Credit Model

Credit Models

José Armando Melchor Soto – 745697
Rolando Fortanell Canedo – 744872
David Campos Ambriz – 7444435

13/04/2026

ITESO

Table of Contents

Executive Summary 2

Introduction 3

Business Model Overview 5

1. Interest Rate Formation Analysis 8

1.1 Base Rate 8

1.2 Inflation Premium 9

1.3 Credit Risk Premium 10

1.4 Liquidity Premium 12

1.5 Administrative Costs 12

1.6 Profit Margin 14

1.7 Risk factors and how they are incorporated into the final rate. 14

2. Credit Risk Modeling Framework 17

2.1 Data & Methodology 17

2.1.1 Dataset Description 17

2.1.2 Description of Variables 18

2.1.3 Class Distribution of the Target Variable 19

2.1.4 Variable Type 20

2.1.5 Data Cleaning 21

2.1.6 Preprocessing 21

2.2 Feature Analysis 22

2.3 Feature Importance 25

2.3.1 XGBoost Hyperparameters 27

2.3.2 SHAP Analysis 27

2.4 Feature Selection 29

2.5 Model Architecture & System Design 32

2.5.1 Architecture 32

2.5.2 System Design 33

3. Results & Model Performance 45

3.1 Model Evaluation 45
3.1.1 Performance Evaluation Results 46
3.1.2 ROC - AUC 47
3.1.3 Confusion Matrix 48
3.1.4 Calibration Curve 49
3.1.5 Predicted PD Distribution 50
3.1.6 Risk Bucket Analysis 52
Model Limitations & Assumptions & Future Improvements 53
Final Discussion & Conclusions 55
References 57
Appendices 59
Appendix A – Main Script 59
Appendix B – Visualization Script 60
Appendix C – Data Preparation Script 63
Appendix D – Data Splitter Script 65
Appendix E – Base Model Script 66
Appendix F – Business Logic Script 68
Appendix G – Classification Model Script 70
Appendix H – Configuration Script 73
Appendix I – Model Evaluation Script 74
Appendix J – Model Script 78
Appendix K – Risk Calculator Script 80
Appendix L – Prints Script 83
Appendix M – Visualization Script 86

Executive Summary

The continued expansion of unsecured personal lending in the United States—with outstanding balances reaching $276 billion by the end of 2025—presents financial institutions with a significant risk management challenge, as the absence of collateral makes borrower evaluation the primary mechanism for controlling credit losses. In response, this project develops a comprehensive credit model for unsecured personal loans by integrating business model analysis, interest rate formation, and quantitative credit risk assessment into a unified framework. The business analysis examines the operational structure of personal lending and explains how the fixed-term, unsecured nature of the product requires institutions to rely heavily on risk-based pricing and borrower creditworthiness assessments. Building on this foundation, the report decomposes the lending rate into its principal components—base rate, inflation premium, credit risk premium, liquidity premium, administrative costs, and profit margin—demonstrating how each contributes to the final loan price and reflects both market conditions and institutional cost structures.

To support underwriting and pricing decisions, a machine learning credit risk model was implemented in Python through a modular object-oriented pipeline designed to automate the borrower evaluation process. A Random Forest classifier was selected as the final predictive model and used to estimate borrower-level Probability of Default (PD), which was subsequently integrated with Loss Given Default (LGD) and Exposure at Default (EAD) assumptions to calculate Expected Loss across the portfolio. These outputs were further incorporated into a business logic framework that enables approval and rejection decisions, borrower risk segmentation, and customized risk-based pricing. Model validation demonstrated strong predictive capability, achieving a test ROC-AUC of 0.93 and robust classification performance across evaluation metrics, indicating reliable discriminatory power and probability estimation. The resulting framework illustrates how predictive analytics and traditional financial risk theory can be combined into an interpretable and operational credit system capable of supporting underwriting decisions, risk-adjusted pricing strategies, and portfolio-level credit risk management within the unsecured lending market.

Introduction

Credit as a consumer product is a tool that has evolved over time and expanded its reach across various markets, helping to finance individuals and allowing them to cover needs that would otherwise be impossible to sustain through simple savings. Within the vast market of credit products, personal loans stand out as one of the most accessible instruments for any person, precisely because their approval depends on the borrower's financial profile rather than on the existence of a physical asset as guarantee. For this reason, the high demand for loans forces credit institutions seeking to participate in this market to maintain more rigorous terms regarding risk management; in the absence of collateral to ensure the company's health during the contract's duration, the decision to issue or deny a loan must ultimately be based on a study of the applicant's creditworthiness.

In this project, the object of study is the unsecured personal loan within the U.S. context, where at the close of last year, this product represented an accumulated balance of $276 billion dollars distributed among more than 26 million active loan accounts, becoming one of the credit products with the greatest expansion in the country in recent years (Schulz, 2026; TransUnion, 2026). This growth reflects both the increasing demand from borrowers seeking flexible financing alternatives and the willingness of lenders to serve a broader range of credit profiles.

The reason for choosing this product lies in the peculiarity of its structure. It is an unsecured product, similar to credit cards, but with the distinct characteristic of having a fixed-term payment structure, which forces its risk assessment to depend exclusively on the applicant's observable variables. This condition makes it a strong case study because, as there is no underlying asset backing the operation, the quality of the evaluation model is the only real protection mechanism available to the issuing institution.

That being said, the general objective of this project is the construction of a comprehensive credit model focused on personal loans. The work begins with the definition of the business model logic and the product's operational structure, followed by an analysis of interest rate formation where each of its components is identified and justified based on U.S. market data. Building on this foundation, a quantitative estimation of credit risk is developed to support

the decision to grant or deny the loan, giving priority to the construction of an interpretable and economically coherent model capable of estimating the Probability of Default (PD) for each applicant and integrating it into an Expected Loss calculation that supports both the approval decision and the pricing of the credit. For the development of this work, 17,523 records corresponding to personal loan products were used, obtained from the Realistic Loan Approval dataset available on Kaggle.

Business Model Overview

The operability of personal loans is based on two structural conditions that mark the risk dynamics as their main rationale in pricing logic. The first is the very nature of installment credit, which means that the borrower receives a certain lump sum of capital that must be repaid in fixed periodic payments, generally between 12 to 84 months. And the second, the "unsecured" condition which, unlike an auto or mortgage loan, means the lender does not have a defined asset that can be taken as collateral in the event of default. Therefore, this "absence" of collateral ends up forcing credit institutions to restructure their cost strategy to adopt a new risk control strategy over the loan. In view of this, the World Bank defines the lending rate as an instrument generally used for short and medium-term financing that is granted according to the customer's creditworthiness and financing objective, adding the terms and conditions of each institution (World Bank, 2024).

The personal loan within the U.S. market acts under the same idea of a fixed-term credit, where the structure of installments or periodic payments gives the bank the ability to project cash flows with greater security, unlike credit cards where one of the greatest risks is the uncertainty regarding payment compliance. According to the Consumer Financial Protection Bureau (or CFPB), in a loan of this type, the issuer is responsible for delivering the totality of the funds at the beginning and the person receiving the loan is committed to repaying them in fixed amounts over an agreed period (Consumer Financial Protection Bureau [CFPB], 2025); this amount received can be used for various purposes depending on the person's needs, which can range from a major purchase, unexpected expenses, liquidating an existing debt, or for some medical or school plan.

This dependence on the "verdict" of a person's ability to pay necessitates a differentiated pricing structure or "risk-based pricing," where both the rate and the amount lent are personalized and adjusted to each client's profile. This profile is generally generated based on credit history, income level, the existence of unresolved debts, and, of course, the applicant's credit score. The following diagram illustrates part of the operational flow of such a credit product, from its application and opening to its closure.

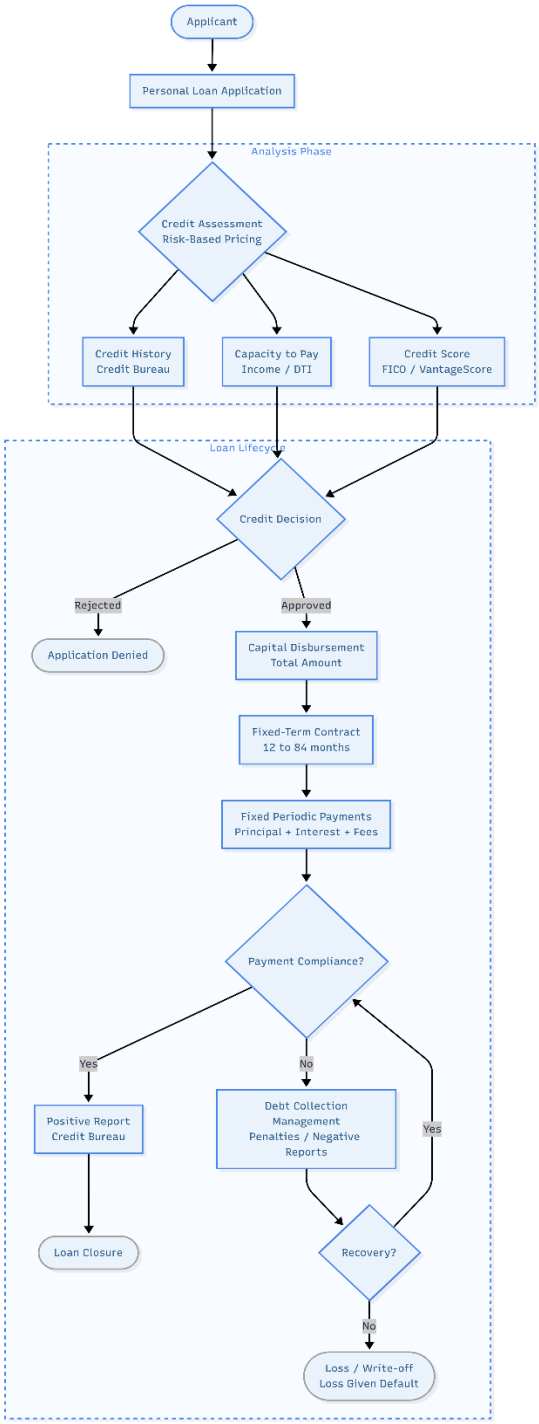

Figure I: Personal Loan Business Model Diagram

As shown in the previous figure, the initial process begins when a person opens a loan application, which triggers a credit evaluation process by the issuer regarding the client, considering factors such as those previously mentioned (credit history, repayment capacity measured by the debt-to-income ratio, and credit score), with the objective of making the decision to issue the loan or not, as well as establishing the conditions and parameters of the credit to be granted. Once the client's application has been approved, the capital is transferred in a single lump sum after having signed the loan contract; in this way, an agreement is formalized regarding the term, periodic installments, the corresponding interest rate, and any other charges or details that may exist, such as commissions. Finally, during the life of the loan, monitoring of the client's compliance or default on payments will be maintained to simultaneously provide feedback to their credit history.

Returning to a market perspective, in recent years the personal loan as a credit product has gained significant relevance in the U.S. market. Just at the close of the final quarter of 2025, the U.S. population has maintained an accumulated debt balance of $276 billion dollars, representing a 10% increase compared to the same quarter of the previous year and being, in turn, the highest figure in the last 20 years (Schulz, 2026). This growth is also consistent with the projections published by TransUnion for 2026, where an 11.2% increase in applications for unsecured personal loans is estimated, which would now make them, along with mortgage credits, the two main credit products with the greatest expansion and consumption (TransUnion, 2026). Furthermore, regarding the structure of demand, there is evidence that more than half of the clients who receive a credit or personal loan (at least 51.4%) end up using these funds to refinance previous debts or pay off pending credit card balances, while the next most common purposes are the payment of everyday expenses (10.8%) and home improvements at 6.6% (Schulz, 2026). This pattern demonstrates that the personal loan frequently tends to be used more as a financial reorganization tool to manage expenses and not necessarily as a last-resort economic subsidy.

This idea of the variety of profiles among applicants is reflected in the distribution of rates according to the person's profile. For example, by the end of 2025, the rate difference between a client with a good score and a poor credit score was more than 1,500 basis points (Schulz, 2026). This demonstrates how the cost of credit is adjusted in proportion to the risk perceived

by the credit issuer in compensation for the absence of collateral. This idea is maintained and reflected in the previous diagram, where credit evaluation is the most important process in the risk control of this credit product.

1. Interest Rate Formation Analysis

The determination of an interest rate for a loan is not an ambiguous process, nor is it the result of an arbitrary decision by the credit issuer; rather, as previously mentioned, it is a quantitative way of expressing the risk and cost that the institution must assume, and it serves to establish strategies that can compensate for these factors to operate sustainably over time. Therefore, each existing layer in the creation of an interest rate represents a source of exposure or cost that the issuer, in a way, transfers to the price of the credit granted. For a credit product such as the personal loan in the United States, its structure is composed of the following components.

1.1 Base Rate

This refers to the federal funds rate, which is considered the main component of any interest rate in the United States credit market and is established by the Federal Open Market Committee (FOMC) of the Federal Reserve. This represents the cost at which banks lend reserves to each other on a daily basis and serves as the primary short-term reference for institutions to set their rates (Board of Governors of the Federal Reserve System, 2025).

In the case of personal loans, this determines the marginal cost at which a bank can fund itself; that is, it represents the floor below which no loan rate can remain active without resulting in losses. For example, after maintaining the rate in a maximum range of 5.25% and 5.50% between July 2023 and September 2024, the FOMC began to reduce interest rates with three consecutive cuts in the second half of 2024 (Board of Governors of the Federal Reserve System, 2025); and, by the close of 2025, the rate stood between 3.5% and 3.75%, which has remained stable through the first quarter of 2026, with the average effective rate for January and March being 3.64% (Federal Reserve Bank of St. Louis, 2026).

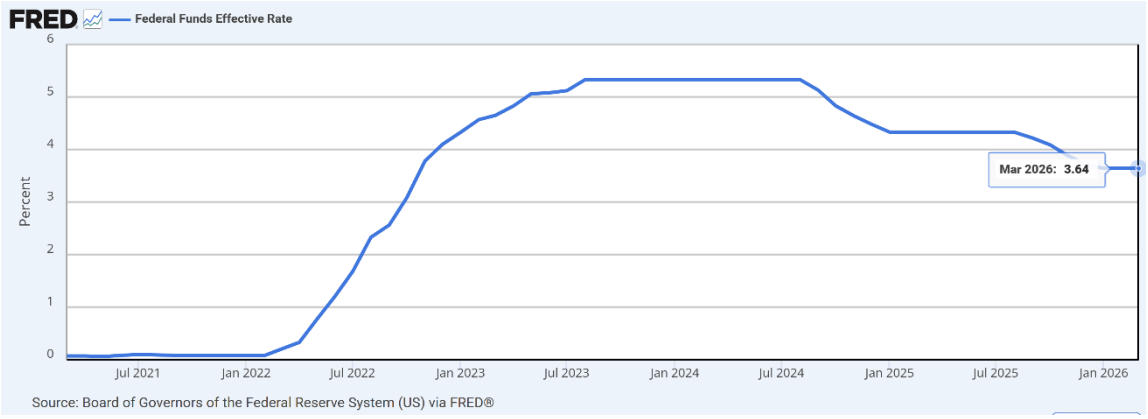

Figure II: Evolution of the Effective Federal Funds Rate 2021-2026

Within this context, Figure 2 provides a visual representation of the scenario that personal loans are undergoing given the current monetary policy. It captures how the reference rate shifted from one of its lowest historical levels to its highest point in years within just a few years, finally descending to a more stable level.

This variation or volatility in the rate involves a lag ranging from weeks to months in the rates offered for a loan, adding the risk that, unexpectedly, the cost of credit changes before the loan is liquidated or renegotiated, affecting both the borrower and the financial institution. According to the Fed's G.19 report, the average rate on 24-month personal loans at commercial banks was 11.65% in November 2025 (Board of Governors of the Federal Reserve System, 2026), implying a total spread of approximately 800 basis points over the reference rate. These points are distributed among some of the remaining components of this part of the analysis.

1.2 Inflation Premium

This represents the compensation the issuer needs to maintain its real purchasing power of the lent capital during the term of the loan (Federal Reserve Bank of San Francisco, 2003). In a personal loan, when a credit institution grants an amount, it is inevitably exposed to and affected by the accumulated inflation during the period the loan is active; therefore, if a loan is granted and the nominal rate does not incorporate compensation for the expected loss due to inflation, the return on that exercise may become negative for the lender.

In the United States, the Consumer Price Index for All Urban Consumers (CPI-U), published monthly by the Bureau of Labor Statistics (BLS), is taken as a reference to establish this inflation premium. According to data from this institution, the CPI-U recorded an annual increase of 2.7% during 2025, with pressures concentrated in food (3.1%), energy (2.3%), and medical services (3.2%) (Bureau of Labor Statistics, 2026a).

By February 2026, annual inflation stood at 2.4%, and core inflation at 2.5% (Bureau of Labor Statistics, 2026). These same levels directly impact the setting of a rate for personal loans for two reasons. First, as inflation is currently above the Fed's long-term 2% target, credit institutions are forced to maintain a positive inflation premium within the rates they grant. Second, given that a personal loan can range from 12 to 84 months, this implies that the inflation premium must include inflationary expectations throughout the entire extension of the contract, in addition to considering current inflation. In other words, if an issuer does not consider that the risk over time could rise, it would be assuming a real interest rate risk without insurance or compensation to protect it.

This structural market condition gives importance to the inflation premium because, unlike mortgage credits, whose rates can be structured as variable and adjusted periodically to the inflationary environment, in the case of personal loans, the credit institution ultimately remains exposed to the differential between expected inflation and realized inflation throughout the life of the credit. Consequently, the inflation premium tends to be higher than that of some variable-rate instruments, which explains why the rate for this product differs from other types of credit.

1.3 Credit Risk Premium

It represents the compensation required by an institution for the probability that the client will default on the credit payment (Consumer Financial Protection Bureau, 2024). In this case, due to the absence of collateral, its calculation as credit risk translates into an additional cost that issuers incorporate into the price of the credit product. As previously mentioned, Risk-Based Pricing is applied as a general rule in the U.S. market when issuing a credit product, and it manifests when an issuer offers higher rates to clients whose credit history is

not as favorable. In this sense, lenders frequently assign higher rates to profiles they consider to be of higher risk, including those with recent bankruptcies, job loss, or delays in their obligations (Consumer Financial Protection Bureau, 2024).

In the case of personal loans, LendingTree data from the last quarter of 2025 shows that clients whose credit score was higher than 720 were granted an average loan of $20,236 dollars at an average rate of 15.08%, while, in the worst-case scenario, those clients with a score below 560 points obtained loans averaging only $11,447 dollars at an average rate of 30.40% (Schulz, 2026). This dispersion in rates is fully presented in the following table.

Credit score range	Avg. APR	Avg. loan amount
720+	15.08%	$20,236
680-719	23.46%	$17,475
660-679	27.20%	$14,195
640-659	28.97%	$12,615
620-639	30.30%	$11,973
580-619	31.10%	$11,486
560-579	31.84%	$11,187
Less than 560	30.40%	$11,447

Source: LendingTree user data on closed personal loans for Q4 2025. Note: Analysis includes loan amounts of $5,000 to $54,999 and repayment terms of 36 to 83 months.

Table I: Average APR and average loan amount by credit score range, Q4 2025. Source: LendingTree Statistics 2026

Another foundation of this premium is based on the observed default rates in personal loans; for example, according to the TransUnion fourth-quarter 2025 report, the delinquency rate for 60 or more days in unsecured personal loans stood at 3.99%, the highest rate since 2023 and the period with the greatest increase in the subprime segment (TransUnion, 2026). This figure is considerably high considering that mortgage delinquency recorded in the same period was only 1.51%, as was the case for auto loans. Again, the absence of an asset that can be held as collateral to be liquidated in the event of a payment default explains the need for issuers of this type of loan to mitigate losses by adjusting their rates and income to compensate for potential losses in delinquent credits.

1.4 Liquidity Premium

This component represents additional compensation that the issuing institution charges in the price of the credit in order to prevent or mitigate the impact on the risk involved in committing long-term capital to an instrument that cannot be liquidated early without a loss (Vipond, 2020). In other words, when a bank grants a personal loan for a certain period, in one way or another, it is losing the ability to reallocate that same capital into other opportunities that may arise during the loan period which could offer higher returns to the lender. Therefore, in addition to the natural cost that banks cover to maintain their static reserves for liquidity against cash deposits, when it comes to personal loans, this additional opportunity cost that the institution acquires is ultimately transferred as an extra premium on the rate (Board of Governors of the Federal Reserve System, 2026).

One way to measure how institutions manage the spread between what they pay for their funds and what they charge for their loans (including premiums) is through the net interest margin (NIM) of the U.S. banking industry. According to the Federal Deposit Insurance Corporation (FDIC, 2025a), the industry NIM in the fourth quarter of 2024 stood at approximately 3.25%, a level similar on average to the pre-pandemic period. Meanwhile, for the second quarter of 2025, this same rate remained at a similar level of 3.26% (FDIC, 2025b); which provides certainty that banks have, in a way, had to adapt and use these premiums to achieve stability in their margins while maintaining a positive differential in favor between what they charge and pay for their loans.

In the specific case of the personal loan, this liquidity premium tends to be higher than in shorter-term products, such as 12-month loans, or auto loans (due to the existence of collateral) and lower than in 30-year mortgage credits, where the risk due to exposure time is considerably greater (FDIC, 2025c).

1.5 Administrative Costs

As the name suggests, this component corresponds to all those administrative costs that the bank must bear throughout the life cycle of the personal loan (Blokhin, 2026). Costs such as

the credit evaluation process, credit bureau inquiries, payment processing costs, statement creation, collection management, and client communication, among others.

In the personal loan market, these costs tend to be higher in proportional terms than in larger-amount products, such as mortgages, because the average loan is relatively small, which leaves little flexibility in how the costs of this exercise must be distributed. For example, according to data from the Fed, by the close of the last quarter of 2025, the average amount of a personal loan was $11,699 dollars (Board of Governors of the Federal Reserve System, 2026). This sum would imply that the costs of processing the loan would diminish a large portion of the potential income, leaving a smaller profit than what could be obtained from a mortgage where the average credit was $368,729, where costs could be distributed more efficiently (TransUnion, 2026).

Due to this same situation, it is common to see that in the U.S. market, the institutions that have gained significant market share and frequently offer this type of loan are FinTechs, given that their origination costs tend to be lower due to the nature of being a purely digital and automated process. According to TransUnion data (2026), by the third quarter of 2025, FinTechs represented 42% of unsecured personal loan issuances, whereas on the same date a year earlier, this figure was one-third of what it is now.

To translate this cost structure into something applicable for the purposes of this project, the operational efficiency record will be taken as a parameter to approximate the administrative costs of the banking industry. According to the FDIC's first-quarter 2025 quarterly report, the non-interest expense of insured institutions amounted to $149 billion dollars, with an efficiency ratio—defined as non-interest expense over net operating income—of 56.2% (Federal Deposit Insurance Corporation, 2025d). That is, for every dollar the issuing institution obtains as income from these exercises, approximately 56 cents are focused on covering operating costs, prior to accounting for possible losses or generated returns. When applied to personal loans, whose costs can be higher than those of other credit products, it is estimated that in the United States, the administrative component incorporated by institutions into their rates for a loan of this type is on average 2% annually over the credit balance (Diette, 2000).

1.6 Profit Margin

Finally, this component is what the institution, regardless of its line of business, incorporates as the profit margin to ensure that its activities can continue operating and that the business remains profitable in the long term (Segal, 2026).

For this type of market, the standard indicator to measure the final performance margin is the return on assets (ROA), which establishes a relationship between total assets under management and net income. According to the FDIC (2025a), by the close of 2024, this metric stood at 1.12%, with a net income of $70.8 billion dollars, and for the year 2025, it was 1.16% (FDIC, 2025a). This stability in levels is part of a trend above the historical long-term average and represents that, approximately, for every dollar the institution managed, it retained 1.16 cents in net profit after meeting all its costs.

Applying this to personal loans, this profit margin is provided by the spread between the reference rate and the final rate charged to the client; this surplus is retained by the bank and is ultimately absorbed by it after having covered all its costs. Therefore, since the ROA generally tends to remain stable, the profit margin for this type of credit ends up being an almost fixed parameter with little adjustment, given that the bank's interest is to protect this component while adjusting others to keep its income stable against variables it cannot control—external macroeconomic changes. This could explain why, with the Fed's five consecutive rate cuts between September 2024 and December 2025, there was a reduction in the average annual rate of personal loans from approximately 12.8% to 12.26%, a 54-basis-point drop, while the reference rate fell by up to 175 basis points in the same period (Kissell, 2026).

1.7 Risk factors and how they are incorporated into the final rate.

The findings above show how individual components allow us to see not only their theoretical meaning but also the logic and reasoning behind their implementation in any credit product, regardless of whether it is a personal loan or any other credit product. However, even though they can be defined in the same way, they maintain certain differences

in their final application when comparing the consumer credit rate offerings in the United States. Taking data from the FED's G.19 report as a reference, by the end of 2025, the average rate on 24-month personal loans at commercial banks was 11.65% (currently 11.40%), while the credit card rate for all accounts was 20.97% (currently 21.00%), and the rate on 48-month auto loans stood at 7.53% (currently 7.36%) (Federal Reserve Bank of St. Louis, 2026a, 2026b, 2026c).

Figure III. a : Comparison of average rates between credit products: 24-month personal loan

Figure III. b : Comparison of average rates between credit products: 48-month auto loan (New car)

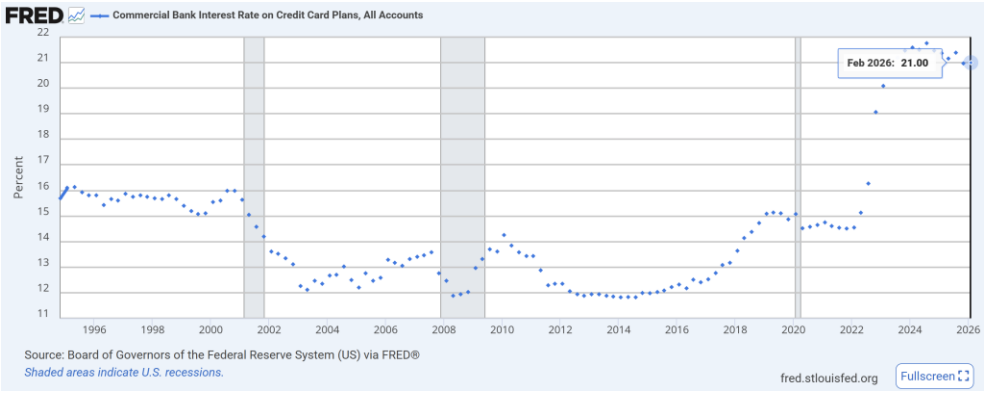

Figure III. c : Comparison of average rates between credit products: all credit card accounts

As can be seen, the behavior of these rates over time has shown a relationship similar to that described in the previous analysis, where the difference between personal loans, auto loans, and credit cards is the accumulation of the risks previously analyzed. In this case, based on the plotted path, it could be said that the personal loan is a midpoint between the products shown, as it is not as extreme as credit cards but not as low as auto loans.

The logic behind this can be explained by the nature of the operational risk of the market in which each product operates. First, credit cards are a revolving credit instrument; that is, a product that allows the client to request loans repeatedly, at any time, and receive them automatically up to a certain limit (Gibson, 2025). In other words, by issuing the card, the institution is permanently exposed (as long as the contract is active) to granting an amount that it may not recover later, since the balance used by the client can vary at any time and there is no fixed amortization schedule. This indeterminacy in flows introduces considerably higher operational and market risk than that of a fixed-term loan, where payments, the amount, and the term are agreed upon from the beginning.

Additionally, in most cases, the credit card is a variable-rate instrument directly linked to the prime rate, which makes it sensitive to macroeconomic movements and forces the bank to maintain an additional premium for this risk (CNBC, 2025). Therefore, despite the fact that both this product and personal loans are not backed by an asset as collateral, the credit card carries a higher operational and market risk premium, which explains a difference of approximately 930 basis points compared to the personal loan in November 2025.

In the case of auto loans, as anticipated, their rate is considerably lower since the existence of collateral provides a final protection to the institution. This "protection" is the bank's right to recover and liquidate the asset in question to reduce the loss acquired in the event of default, which significantly reduces its "LGD" (Loss Given Default) and allows for the reduction of the credit risk premium to levels that other products, such as the personal loan or a credit card, cannot reach, as they would cease to be profitable. Coupled with this, FDIC data on 60-day delinquency in auto loans was 1.50% in the fourth quarter of 2025, compared to the 3.99% recorded in unsecured personal loans (TransUnion, 2026), confirming that the rate differential reflects a real difference that can be measured from each client's expected

loss profile; therefore, based on the expected risk, a bank can afford to offer the same person higher or lower rates depending on the product they request.

Finally, regarding regulatory risk, although it does not manifest as directly in the construction of the product as in the previous cases, it ultimately manifests within the operation by affecting the cost structures for the issuing institution. In the case of loans, compliance with the Risk-Based Pricing Rule under the FCRA, the CFPB's supervision regarding origination practices, and the obligation to meet the capital requirements set by the FDIC and the Federal Reserve are more intensive than in auto credit products and imply compliance costs that institutions must incorporate into their operating margins to maintain certain profitability and security in their products (Consumer Financial Protection Bureau, 2024).

2. Credit Risk Modeling Framework

2.1 Data & Methodology

2.1.1 Dataset Description

The dataset used in this project corresponds to the *Realistic Loan Approval Dataset (US & Canada)* obtained from *Kaggle*. This dataset was selected due to its realistic structure and its similarity to the criteria commonly used by banks and financial institutions in credit risk assessment.

According to its description and an exploratory analysis, the dataset incorporates realistic correlations and business logic, making it suitable for modeling real-world credit decision processes; this is particularly important for the objective of this project, as it allows the developed model to better approximate actual lending behavior.

The dataset contains approximately 50,000 financial records prior to data cleaning, where each observation represents an individual loan applicant, making it well-suited for supervised machine learning in a binary classification setting with realistic patterns.

However, despite these advantages, the dataset presents some limitations: it relies on a simplified approval logic compared to real-world banking systems that typically use hundreds of variables, it lacks a temporal component and therefore does not capture changes in borrower behavior over time, it assumes a single currency (USD) without considering exchange rate risk, and it does not incorporate external macroeconomic factors such as interest rates, inflation, or economic conditions; consequently, the results should be interpreted as a simplified approximation of real credit risk environments rather than a fully production-ready system.

In its raw form, the dataset has the following structure:

- Observations: 50,000 records
- Initial variables: 20 columns, including identification fields and the target variable.
- Post-preprocessing variables: 19 total columns, composed of:
    - – 17,523 records
- Target Variable: *loan_status* (0/1)

The reduction in the dimensionality of the dataset is due to the filtering of the *product_type* variable, where only observations corresponding to *Personal Loan* were retained. As a result, all records associated with *Credit Card* and *Line of Credit* were removed to ensure consistency in the analysis and to focus on a single credit product. Additionally, the *customer_id* column was dropped, as it does not provide predictive value for the model and could introduce noise into the learning process.

2.1.2 Description of Variables

Table # presents the complete variable dictionary of the credit dataset. One column (*customer_id*) was removed and excluded from the analysis, as it does not provide predictive value for the modeling process and may contain personally identifiable information (PII).

Variable	Type	Description	Range / Values
age	Numerical	Age of the applicant	18 - 70
occupation_status	Categorical	Employment or occupational status of the applicant	Categories
years_employed	Numerical	Number of years the applicant has been employed	≥ 0
annual_income	Numerical	Annual income of the applicant	> 0
credit_score	Numerical	Credit score representing borrower creditworthiness	375 - 850
credit_history_years	Numerical	Length of credit history in years	≥ 0
savings_assets	Numerical	Total savings and assets owned by the applicant	≥ 0
current_debt	Numerical	Total current outstanding debt	≥ 0
defaults_on_file	Binary	Indicates whether the applicant has past defaults	0 / 1
delinquencies_last_2yrs	Numerical	Number of delinquencies in the last 2 years	≥ 0
derogatory_marks	Numerical	Number of negative credit records	≥ 0
product_type	Categorical	Type of financial product requested	Personal Loan
loan_intent	Categorical	Purpose of the loan	Categories
loan_amount	Numerical	Requested loan amount	> 0
interest_rate	Numerical	Interest rate assigned to the loan (%)	8% - 17%
debt_to_income_ratio	Numerical	Ratio of debt relative to income	0.002 - 0.8
loan_to_income_ratio	Numerical	Ratio of loan amount to income	0.015 - 2.001
payment_to_income_ratio	Numerical	Ratio of payment obligations to income	0.015 - 0.667
loan_status	Binary (Target)	Loan approval decision	0 / 1

Table II. Description of Model Variables and Data Types

2.1.3 Class Distribution of the Target Variable

Table # summarizes the distribution of the target variable (*loan_status*). The dataset exhibits a moderate class imbalance, with the *Approved* category representing approximately 52% of the observations and the *Rejected* category accounting for 48%. This indicates a slight dominance of the Approved class over the Rejected class, although the distribution remains balanced.

While the imbalance is not severe, it can still influence model performance, particularly if evaluation relies solely on accuracy, as the model may become biased toward predicting the majority class. In the context of credit risk, this is especially relevant since different types of misclassifications carry different costs.

For this reason, performance metrics that account for class imbalance, such as F1-score and ROC-AUC, are more appropriate for assessing the model, as they provide a more reliable evaluation of its predictive capability.

Distribution of the Target Variable 17,523

Credit Model Classification — Training Set observations

Class	Proportion (%)	Frequency	Proportion
Approved *Majority class*	52.0%	9,106 Records	0.5197
Rejected *Minority class*	48.0%	8,417 Records	0.4803

■ Approved ■ Rejected

Figure IV. Distribution of the Target Variable

2.1.4 Variable Type

The dataset variables can be grouped into three main categories:

Numerical variables (6): *years_employed, credit_history_years, interest_rate, debt_to_income_ratio, loan_to_income_ratio, payment_to_income_ratio.*

Discrete numerical variables (9): *age, annual_income, credit_score, savings_assets, current_debt, defaults_on_file, delinquencies_last_2yrs, derogatory_marks, loan_amount.*

Categorical variables (2): *occupation_status, product_type, loan_intent.*

It is important to note that several variables initially presented inconsistencies in their data types, as some numerical features were stored as object (string) due to the presence of noise

characters. This required explicit data cleaning and type conversion during the preprocessing stage to ensure consistency and compatibility with machine learning models.

2.1.5 Data Cleaning

The data cleaning process was minimal, as the dataset was already well-structured and exhibited high data quality. As mentioned in previous sections, an initial inspection confirmed the absence of missing values across all variables, eliminating the need for imputation techniques. Additionally, no significant inconsistencies, invalid entries, or extreme outliers were identified that required correction or removal, which indicates that the dataset was designed to closely resemble a clean and production-ready environment.

The cleaning procedures were implemented within a *Jupyter Notebook*, where exploration analysis and validation steps were conducted to ensure data integrity prior to modeling. The main cleaning steps performed were focused on improving the relevance and consistency of the dataset. First, the *customer_id* variable was removed, as it does not provide predictive value for the model and may introduce unnecessary noise. Furthermore, the dataset was filtered to retain only observations corresponding to *Personal Loan* within the *product_type* variable, removing records associated with *Credit Card* and *Line of Credit*. This step was essential to maintain consistency in the analysis and to focus the modeling process on a single credit product. Given the circumstances, no extensive data cleaning procedures were required. The dataset was considered sufficiently clean for modeling purposes, allowing the workflow to focus primarily on preprocessing, feature engineering, and model development, rather than corrective data treatment.

2.1.6 Preprocessing

The data preprocessing stage focused on transforming the dataset into a structured and model-ready format suitable for machine learning algorithms. This process was implemented following a modular pipeline approach, ensuring clarity, reproducibility, and scalability throughout the project.

First, the target variable (*loan_status*) was separated from the feature set, defining the dependent variable (*y*) and the independent variables (*X*). As part of this step, non-predictive and potentially problematic variables such as interest_rate were removed from the feature matrix to avoid data leakage, since this variable may contain information derived from the approval decision itself .

Next, categorical variables including *occupation_status, product_type*, and *loan_intent* were transformed into numerical representations using one-hot encoding. This transformation allows the model to process categorical information without assuming any ordinal relationship between categories. The encoding was implemented using the *get_dummies* function with the *drop_first=True* parameter, which helps prevent multicollinearity by avoiding redundant dummy variables.

Additionally, the preprocessing stage ensured that all variables were in appropriate numerical formats, following prior cleaning steps where necessary. This guarantees compatibility with machine learning algorithms that require numerical input data.

Finally, the dataset was divided into training and testing subsets using a stratified sampling approach. This ensures that the proportion of the target classes (*Approved* and *Rejected*) remains consistent across both subsets, which is particularly important given the slight class imbalance observed in the data. The split was performed using a 75/25 ratio and a fixed random state to ensure reproducibility of results, these preprocessing steps ensured that the dataset was properly structured, free of data leakage and overfitting risks, and suitable for reliable model training and evaluation.

2.2 Feature Analysis

To ensure that the development of the credit model does not remain purely a statistical exercise, quantitative financial and behavioral variables were incorporated to objectively evaluate the borrower's creditworthiness and estimate the probability of default. This methodology follows the logic of modern retail credit risk assessment frameworks, in which borrower-level demographic, financial, and behavioral indicators are used to statistically model default propensity and support lending decisions.

Borrower financial capacity and indebtedness

This category includes variables related to the applicant's financial strength and debt burden, which are fundamental to assessing repayment ability. Metrics such *as annual_income, loan_amount, debt_to_income_ratio, loan_to_income_ratio,* and *payment_to_income_ratio* provide insight into the borrower's leverage position and ability to sustain additional financial obligations. Higher leverage ratios may indicate overextension and reduced repayment capacity, increasing the probability of default.

Creditworthiness and historical repayment behavior

This dimension groups variables associated with the borrower's historical financial discipline and prior credit performance. Indicators such as *credit_score, credit_history_years, defaults_on_file, delinquencies_last_2yrs,* and *derogatory_marks* help approximate the borrower's historical reliability and repayment consistency. These variables are widely recognized in credit underwriting as strong predictors of future default behavior, as they reflect previous fulfillment or breach of financial obligations.

Borrower profile and stability indicators

This category captures demographic and employment-related variables that may provide indirect signals regarding borrower stability and long-term repayment reliability. Variables such as *age, years_employed,* and *savings_assets* help characterize the applicant's financial maturity, labor stability, and available liquidity reserves. Longer employment tenure and greater savings levels may indicate stronger economic resilience in adverse conditions.

Pricing and lender risk perception

Certain variables also reflect the institution's prior internal perception of borrower risk and loan pricing adjustments. For example, *interest_rate* may embed the lender's preliminary

assessment of applicant risk, where higher assigned rates generally correspond to higher perceived default probability. This variable can serve as a proxy for prior underwriting judgment and market-based risk pricing.

Additionally, to ensure mathematical robustness during the preliminary feature selection stage and avoid multicollinearity issues within the predictive framework, an exploratory correlation analysis was conducted. Correlation and covariance matrices were evaluated to identify redundant predictors and verify the relative independence of explanatory variables prior to model development. This process helps improve model interpretability, statistical stability, and predictive reliability by reducing overlapping information among features.

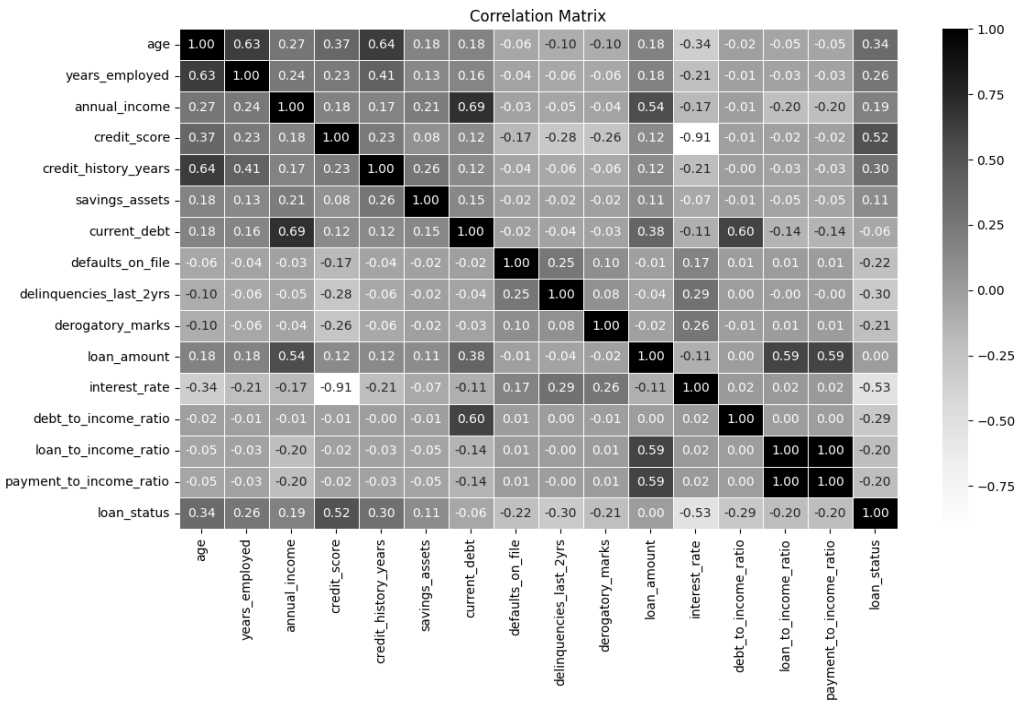

Figure V. Correlation Matrix of All Variables

This process aims to identify variables that capture independent dimensions of the borrower's financial and behavioral profile while providing an initial approximation of the relationship between explanatory variables and the target variable *loan_status*. Through the correlation matrix analysis presented in Figure X ; variables exhibiting stronger relationships with the target such as *interest_rate, credit_score, loan_amount, delinquencies_last_2yrs,* and

credit_history_years appear to have greater explanatory potential within the predictive framework. Conversely, variables showing weaker correlation may indicate a lower linear association with default behavior and therefore potentially contribute less explanatory value in the initial stages of model development.

2.3 Feature Importance

To mitigate potential human bias in manual variable selection and reduce the inclusion of predictors that may introduce noise into the model, a feature importance filtering process was implemented using the gradient boosting decision tree algorithm XGBoost as an additional validation mechanism for variable relevance.

In credit risk modeling, ensemble learning techniques such as XGBoost are widely recognized for their ability to detect nonlinear relationships and complex interactions among explanatory variables that may not be captured through traditional linear statistical methods. Additionally, XGBoost provides a quantitative measure of feature importance based on gain, which evaluates the improvement in predictive performance generated by each variable when used in tree-splitting decisions throughout the boosting process.

By iteratively assessing the contribution of each predictor during model training, this methodology allows for the identification of variables with the highest statistical relevance in explaining borrower default behavior. As illustrated in Figure X, variables such as *credit_score, credit_history_years, debt_to_income_ratio*, and *years_employed* exhibited the highest feature importance scores, suggesting that these predictors contribute most significantly to the classification of borrowers according to their probability of default.

Conversely, variables with lower importance scores, while still potentially informative, demonstrate comparatively weaker marginal contribution to the predictive framework. This process provides an additional layer of statistical validation for feature selection, improving model interpretability and helping ensure that the final predictive model is constructed upon the most informative borrower characteristics.

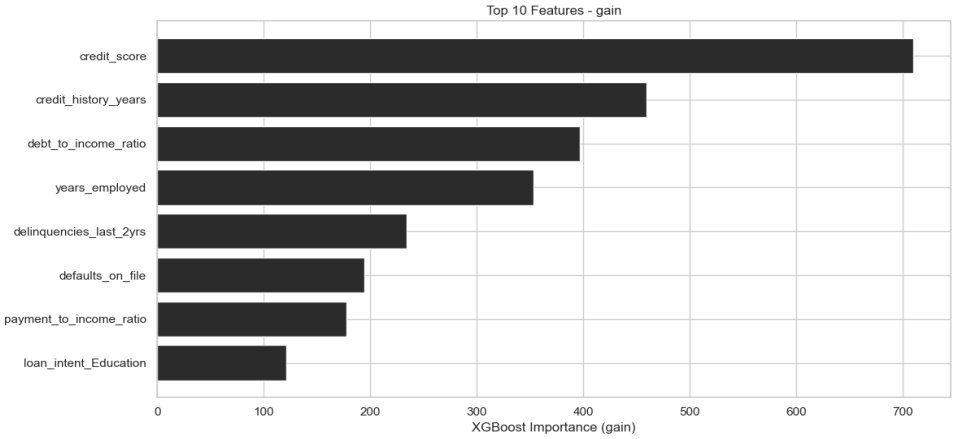

Figure VI. Top 10 Most Important Features Based on XGBoost Gain Importance

Although the results presented in the previous figure provide a useful indication of which variables may be suitable for inclusion in the final model, it is important to note that feature importance metrics only measure the statistical contribution of each variable and do not capture the direction of the relationship with the target variable. In other words, a high level of importance does not necessarily imply an economically meaningful or interpretable relationship. Therefore, it is necessary to further evaluate the direction and economic consistency of these predictors to ensure that their inclusion in the final model can be properly justified and interpreted within a financially coherent framework.

2.3.1 XGBoost Hyperparameters

XGBoost Hyperparameters		
Parameter	**Value**	**Description**
objective	binary:logistic	Specifies a binary classification objective and outputs predicted probabilities for the positive class.
n_estimators	250	Determines the total number of boosting rounds (trees) used during training. More trees can improve learning but may increase overfitting if not properly regularized.
max_depth	2	Sets the maximum depth of each decision tree, controlling model complexity and helping prevent overfitting.
learning_rate	0.01	Controls the contribution of each tree to the final model. Lower values produce slower, more stable learning.
subsample	0.8	Defines the proportion of training observations randomly sampled for each boosting iteration, improving generalization and reducing overfitting.
colsample_bytree	0.8	Specifies the proportion of features randomly sampled for each tree, reducing dependency on specific predictors and improving robustness.
eval_metric	logloss	Measures prediction error using logarithmic loss, appropriate for evaluating binary classification probability outputs.
random_state	42	Fixes the random seed to ensure reproducibility and consistency of training results.

Table III. Selected Hyperparameters Used for XGBoost Training

An XGBoost classifier was trained using the hyperparameters presented below. These parameters were selected to balance model complexity and feature importance stability while reducing the risk of overfitting through controlled learning and subsampling techniques.

The results indicate that a subset of variables contributes more significantly to the prediction of borrower default behavior. These predictors were therefore considered strong candidates for the final model, as they provide the highest information gain and explanatory contribution in predicting *loan_status*.

2.3.2 SHAP Analysis

To interpret the relationships identified by the model, the Shapley Additive Explanations (SHAP) framework was implemented as an interpretability tool for the selected variables. This technique decomposes the predictions generated by the XGBoost model into the additive contribution of each feature, allowing the magnitude and direction of each variable's impact on the final classification outcome to be observed for every individual prediction (Gramegna & Giudici, 2021).

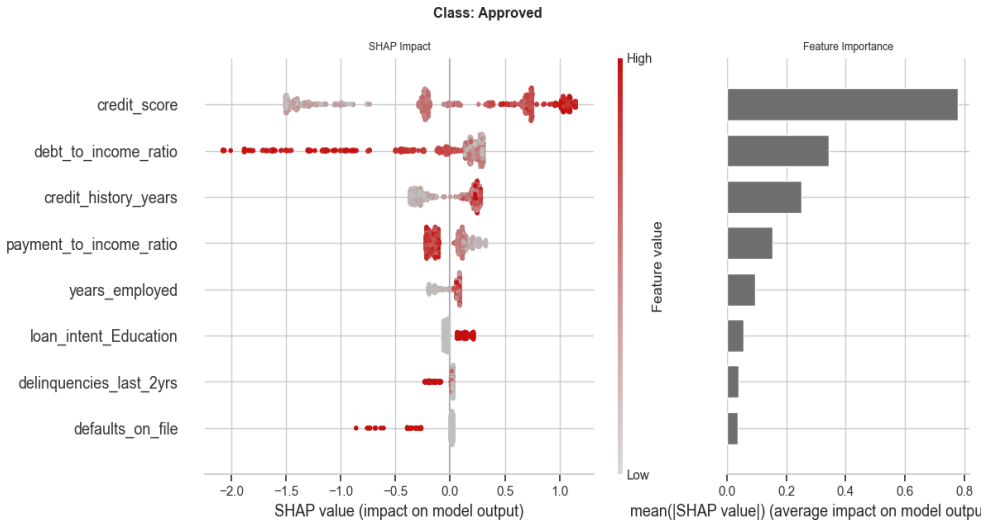

Figure VII. a : SHAP Summary Plot of Feature Contributions for Approved Loan Classification

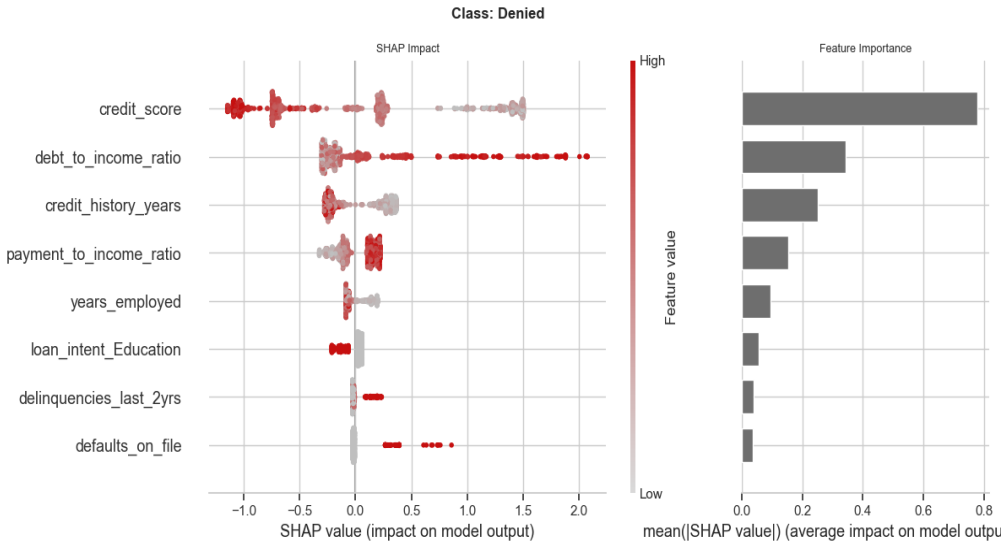

Figure VII.b : SHAP Summary Plot of Feature Contributions for Denied Loan Classification

As shown in the SHAP value distributions for both the Approved and Denied classes (Figures VII.a and VII.b), variables such as *credit_score* and *debt_to_income_ratio* exhibit the strongest directional influence on model predictions. Higher values of *credit_score* contribute positively toward the Approved classification, as indicated by the concentration of high-value observations on the positive side of the SHAP distribution, while lower credit scores shift predictions toward the Denied category.

Conversely, higher values of *debt_to_income_ratio* generate a negative contribution toward loan approval and increase the probability of denial, reflecting the model's penalization of borrowers with greater leverage and repayment burden. A similar directional pattern is observed for *credit_history_years,* where longer credit histories tend to support approval outcomes, suggesting that more established credit experience is associated with lower perceived risk.

Additionally, variables such as *payment_to_income_ratio, years_employed*, and *defaults_on_file* also influence classification, although to a lesser extent. In particular, the presence of prior defaults contributes negatively to approval likelihood and positively toward denial, reinforcing the importance of historical repayment behavior in the underwriting decision process.

2.4 Feature Selection

After completing the theoretical and empirical validation stages through correlation analysis, feature importance, and SHAP interpretability analysis, a final selection of eight variables was determined for the construction of the credit scoring model. This selection aims to preserve a balanced representation of the main borrower risk dimensions while ensuring both statistical relevance and financial interpretability.

Creditworthiness and Historical Repayment Behavior

The following variables were selected:

- *credit_score*
- *credit_history_years*
- *delinquencies_last_2yrs*
- *defaults_on_file*

According to the World Bank Group (2020), historical repayment performance and previous credit behavior are among the most important inputs in the development of credit scoring systems, as they provide lenders with measurable evidence regarding a borrower's past

ability to fulfill debt obligations. In this sense, *credit_score* was selected as a direct proxy of overall borrower creditworthiness, since higher credit scores generally indicate a lower probability of repayment failure.

Similarly, *credit_history_years* was included to capture the maturity and depth of the borrower's credit experience. As stated by the World Bank Group (2020), longer credit histories allow lenders to observe repayment behavior over a broader time horizon, thereby improving the reliability of risk assessment.

Additionally, *delinquencies_last_2yrs* and *defaults_on_file* were incorporated because prior missed payments and historical defaults are commonly interpreted as strong indicators of future repayment distress. According to the Government Accountability Office (2019), delinquency and default records remain among the most influential derogatory events in borrower credit profiles and may significantly affect predicted repayment risk.

Repayment Capacity and Leverage Level

The following variables were selected:

- *debt_to_income_ratio*
- *payment_to_income_ratio*

According to the Consumer Financial Protection Bureau (CFPB, 2013), borrower leverage and debt burden metrics are fundamental in evaluating repayment capacity, as lenders are expected to assess a borrower's debt obligations relative to available income when determining creditworthiness.

Within this framework, *debt_to_income_ratio* was selected because it reflects the proportion of total debt obligations relative to borrower income, serving as a direct measure of financial leverage. Higher debt-to-income ratios may indicate that the borrower has limited residual income available to absorb new obligations or financial shocks.

Likewise, *payment_to_income_ratio* was incorporated to estimate the proportion of income consumed specifically by periodic payment obligations. According to the World Bank Group

(2025), lenders should evaluate not only current indebtedness but also the borrower's broader financial obligations and repayment commitments when assessing credit risk.

Borrower Stability and Socioeconomic Indicators

The following variables were selected:

- *years_employed*
- *loan_intent_Education*

According to the Consumer Financial Protection Bureau (2013), employment status is commonly evaluated during underwriting because it provides an indication of income consistency and financial stability. Accordingly, *years_employed* was selected as a proxy for labor stability, under the assumption that borrowers with longer employment tenure may demonstrate greater income consistency and lower financial uncertainty.

Finally, *loan_intent_Education* was incorporated to capture the stated purpose of borrowing, which may influence borrower risk segmentation. As noted by the World Bank Group (2025), lenders should consider the broader context and intended purpose of borrowing when evaluating borrower obligations and responsible lending practices. Different loan purposes may reflect distinct borrower motivations and repayment patterns, which can influence overall credit behavior.

2.5 Model Architecture & System Design

2.5.1 Architecture

2.5.1.1 Project Structure

Figure VIII. a : Project Architecture and Modular File Structure of the Credit Risk Model

2.5.1.2 Flow Diagram

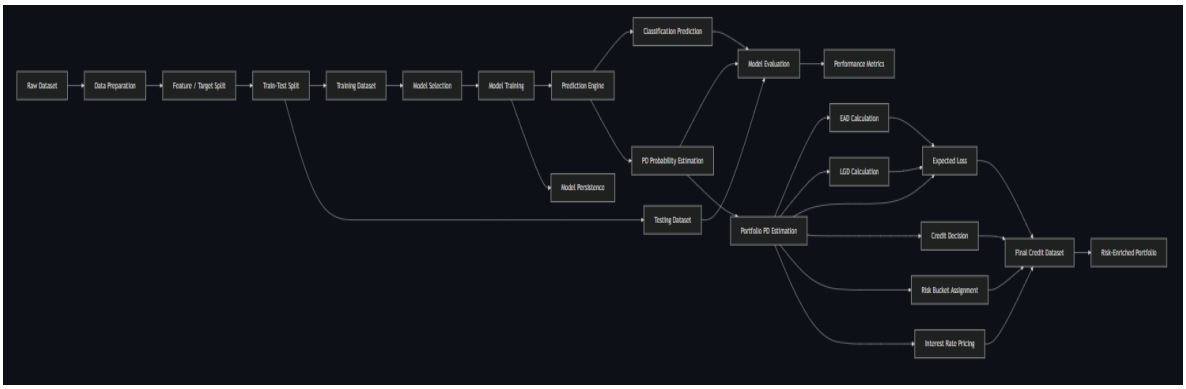

Figure VIII. b : Credit Risk Prediction Pipeline

2.5.1.3 Object-Oriented Design

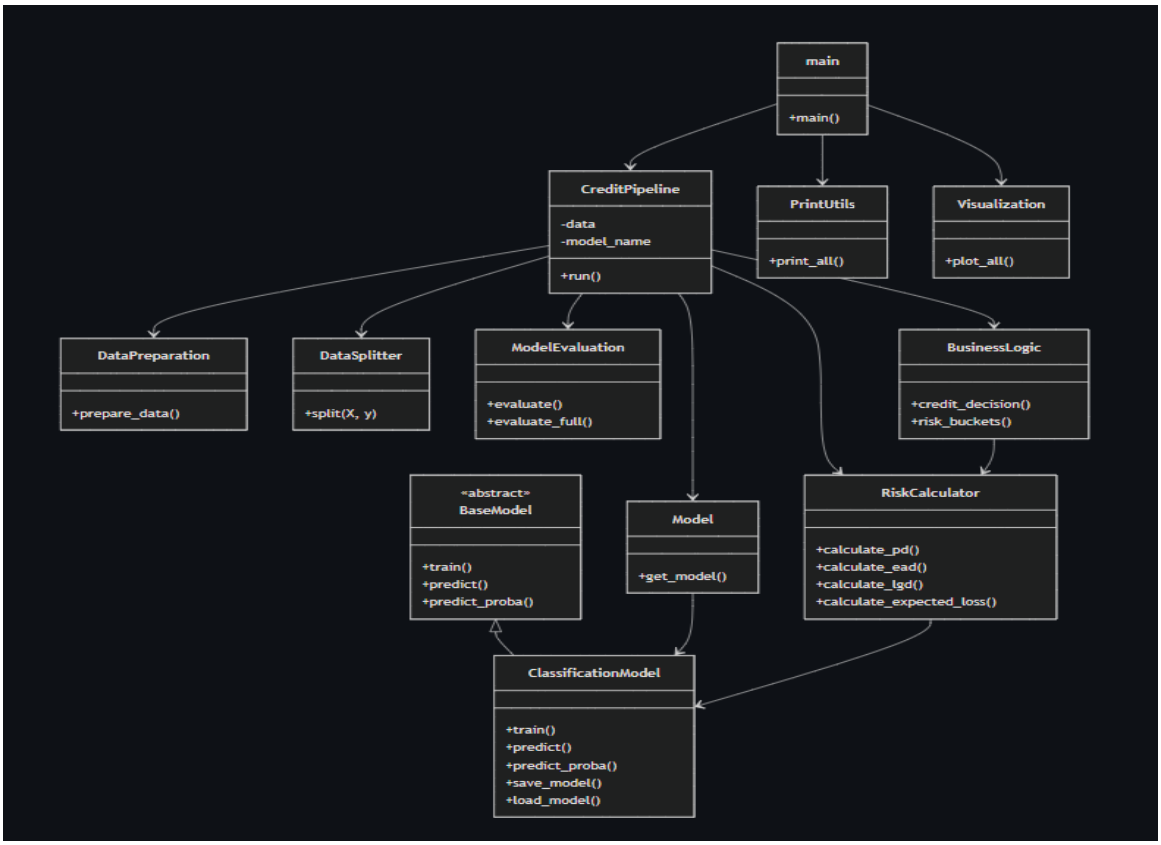

Figure VIII. c : Object-Oriented Design of the Credit Risk Framework

2.5.2 System Design

2.5.2.1 Model Pipeline Overview

Following the feature selection process, a structured credit risk modeling pipeline was developed to operationalize the selected predictive variables into a complete borrower assessment framework. The pipeline was designed as an end-to-end modular system that automates the full credit evaluation process, beginning with raw data preprocessing and concluding with final business decision-making, loan pricing, and portfolio enrichment.

As illustrated in the pipeline diagram, the process begins with the raw dataset, which undergoes a data preparation stage where the information is cleaned, transformed, and structured for modeling purposes. After preprocessing, the dataset is divided into features

and target variables, followed by a train-test split to separate observations used for model training from those reserved for validation.

The training subset is then passed into the model selection and model training stages, where the chosen machine learning algorithm is fitted to historical borrower data. Once trained, the model enters the prediction engine, generating both classification predictions and probability of default (PD) estimations for each observation.

These outputs are subsequently evaluated through the model evaluation module, where predictive performance is measured using classification metrics to assess discriminatory power and overall model robustness. Simultaneously, predicted probabilities are extended to the full portfolio through the portfolio PD estimation process, allowing borrower-level default probabilities to be calculated across the complete dataset.

Following PD estimation, the pipeline computes the core credit risk metrics through dedicated modules for Exposure at Default (EAD) and Loss Given Default (LGD), which are integrated into the Expected Loss calculation:

$$EL = PD \times LGD \times EAD$$

where PD (Probability of Default) represents the likelihood that a borrower will fail to meet debt obligations within the defined time horizon, LGD (Loss Given Default) measures the proportion of total exposure expected to be lost if default occurs after accounting for recoveries, and EAD (Exposure at Default) represents the total outstanding amount exposed to loss at the time of borrower default.

At the business layer, the estimated PD values are further leveraged to support credit decisioning, risk bucket assignment, and risk-adjusted interest rate pricing, enabling the institution to classify borrowers according to their creditworthiness and assign appropriate lending conditions.

Finally, all generated outputs including expected loss, borrower decisions, segmentation labels, and pricing information are consolidated into a final credit dataset, resulting in a risk-enriched portfolio suitable for deployment, portfolio monitoring, and strategic risk management. This integrated framework allows predictive analytics, traditional risk metrics,

and business decision rules to operate cohesively within a single scalable credit risk architecture.

2.5.2.2 Data Preparation Module

The data preparation module transforms the raw borrower dataset into a structured format suitable for model training. This stage ensures that the data is properly organized and standardized before entering the modeling process.

First, the target variable (*loan status*) is separated from the predictor variables, while non-predictive and leakage-prone columns, such as the observed interest rate, are removed from the dataset.

Next, categorical variables are converted into numerical format using one-hot encoding, allowing qualitative borrower information to be incorporated into the machine learning model. To avoid multicollinearity, the first category of each encoded variable is omitted during transformation.

After encoding, the dataset is filtered to retain only the variables previously selected during the feature selection stage, ensuring consistency between feature analysis and final model implementation. A validation check is also performed to confirm that all selected predictors remain available after preprocessing.

This preprocessing framework guarantees that the final dataset remains clean, consistent, and aligned with the model's selected predictive inputs before proceeding to the training stage.

2.5.2.3 Model Development

To ensure scalability, modularity, and ease of experimentation throughout the modeling process, the machine learning framework was designed using an object-oriented architecture that allows multiple predictive algorithms to be integrated under a unified structure.

The implementation begins with an abstract BaseModel class, which defines the core interface that all predictive models must follow, including standardized methods for training, prediction, and probability estimation. This abstraction creates consistency across model implementations and establishes a common structure for future extensibility.

Building on this foundation, a dedicated ClassificationModel wrapper was developed to support multiple classification algorithms within a single reusable class. Currently, the framework enables dynamic implementation of *Logistic Regression, Random Forest*, and *XGBoost*, allowing efficient comparison across alternative classification models without requiring modifications to the broader pipeline.

To improve maintainability and reproducibility, model hyperparameters are centrally managed through a configuration module, where each algorithm's parameters are stored in a structured dictionary format. This design simplifies parameter tuning and reflects practices commonly adopted in scalable machine learning workflows and MLOps frameworks such as MLflow.

Additionally, a factory-based Model class was implemented to dynamically instantiate models according to task type and selected algorithm. While the current implementation focuses on classification-based credit risk models, the framework was intentionally structured to support future expansion into regression tasks if required, enabling adaptation for broader predictive modeling applications.

This modular architecture establishes the foundation for the classification model selection and optimization process presented in the following section.

2.5.2.4 Classification Model Development

Random Forest Hyperparameters

Random Forest Hyperparameters		
Parameter	**Value**	**Description**
n_estimators	1000	Number of decision trees used in the forest. More trees generally improve stability and predictive performance.
max_depth	2	Maximum depth of each tree, limiting complexity and reducing overfitting.
class_weight	balanced	Adjusts class weights inversely to class frequencies to handle class imbalance.
n_jobs	-1	Uses all available CPU cores to speed up model training and prediction.
random_state	42	Ensures reproducibility by fixing the random seed.

Table IIIV. Selected Hyperparameters Used for Random Forest Training

Among the candidate algorithms evaluated during experimentation, a Random Forest classifier was selected as the final predictive model due to its strong performance, robustness to non-linear relationships, and ability to capture complex interactions between borrower characteristics. These properties make it particularly suitable for credit risk applications, where default behavior is often influenced by multiple interacting financial and behavioral factors.

The model was trained using the hyperparameters presented below, which were selected to balance predictive performance, model complexity, and generalization capability throughout the training process. A large number of trees (*n_estimators = 1000*) was used to improve prediction stability and reduce variance across the ensemble, while limiting tree depth (*max_depth = 2*) helps control complexity and mitigate overfitting. Additionally, class balancing techniques (*class_weight = balanced*) were incorporated to address target imbalance and improve classification performance across both default and non-default observations. A fixed random seed was also applied to ensure reproducibility of results.

Once trained, the model generates probability estimates used to predict borrower default behavior, forming the basis for the Probability of Default (PD) calculations within the broader credit risk pipeline.

2.5.2.5 Risk Calculation Framework

Following classification model training, the predicted probability outputs are transformed into traditional credit risk metrics through a dedicated risk calculation framework. This

component translates machine learning predictions into standardized financial risk measures commonly used in credit portfolio management.

First, the probability estimates generated by the classification model are interpreted directly as Probability of Default (PD), representing the estimated likelihood that a borrower will fail to meet debt obligations within the defined time horizon. These probabilities are obtained through the model's probability prediction function and assigned at the individual borrower level.

To complement PD estimation, Loss Given Default (LGD) is incorporated as a fixed assumption of 45%, representing the proportion of exposure expected to be lost in the event of borrower default after accounting for potential recoveries. While simplified, this constant assumption provides a practical approximation commonly used in introductory credit risk frameworks.

Additionally, Exposure at Default (EAD) is estimated using each borrower's total loan amount, serving as a proxy for the outstanding exposure at the moment of default. This assumes that the full loan balance remains exposed if default occurs.

As previously introduced in the pipeline overview, these three components are integrated under the Expected Loss (EL) framework to estimate the projected monetary loss associated with each borrower by combining default likelihood, loss severity, and exposure magnitude. This metric serves as a core measure for portfolio risk assessment and capital allocation within the broader credit risk pipeline

2.5.2.6 Business Logic Integration

The Business Logic layer serves as the operational bridge between the predictive credit risk model and the practical lending decisions of the institution. While the machine learning model estimates the borrower's Probability of Default (PD), predictive output alone does not constitute a complete credit model. Financial institutions must transform these statistical estimates into actionable business decisions regarding loan approval, risk segmentation, and

interest rate pricing. Therefore, the Business Logic framework was designed to replicate the underwriting and pricing process of a lender operating within the U.S. personal loan market.

This structure follows the same risk-based pricing philosophy previously identified in the theoretical business model and interest rate formation analysis, where lenders determine both eligibility and pricing based on the borrower's estimated level of credit risk. Under this framework, applicants with lower expected risk are offered more favorable borrowing terms, while riskier borrowers either face higher interest rates or are denied credit altogether.

Credit Approval Decision Framework

The first function of the Business Logic model is to determine whether an applicant should be approved or rejected for credit. In practical lending environments, institutions establish an internal risk tolerance threshold that defines the maximum acceptable default probability for loan issuance. This threshold reflects the lender's desired portfolio quality, capital strategy, and tolerance for expected losses.

Within the model, this decision rule is structured through a binary approval framework:

$$Decision_i = \begin{cases} 1, & \text{if } PD_i < \theta \\ 0, & \text{if } PD_i \geq \theta \end{cases}$$

where:

- PD_i represents the estimated Probability of Default of borrower i,
- θ represents the institution's internal approval threshold.

For this project, the approval threshold was calibrated at 40%, meaning any applicant with an estimated default probability above 40% is rejected from receiving credit. This threshold-based decisioning mechanism reflects standard underwriting practices in consumer lending, where lenders systematically decline applicants whose projected risk exceeds internal portfolio tolerance. This approach ensures that the institution preserves portfolio quality by preventing the origination of loans to excessively risky borrowers, thereby reducing the concentration of high-default exposures.

Risk Segmentation Framework

After determining approval status, the next stage of business logic involves classifying borrowers into internal risk groups. Financial institutions commonly segment customers into rating categories or score bands to facilitate portfolio monitoring, performance tracking, and pricing differentiation.

To replicate this practice, the model groups borrowers according to their Probability of Default into quantile-based risk buckets. This means the borrower population is divided into equally sized segments ranked by increasing PD:

$$Bucket_i = Quantile(PD_i)$$

Under the implemented framework, borrowers are divided into five risk groups (quintiles):

- Bucket 1: Lowest risk borrowers
- Bucket 2: Low-moderate risk
- Bucket 3: Moderate risk
- Bucket 4: Moderate-high risk
- Bucket 5: Highest risk borrowers

This segmentation mechanism allows the institution to better understand the composition of its portfolio and evaluate how performance varies across risk groups. Additionally, it provides a structured way to analyze the distribution of pricing and approval decisions among different borrower profiles.

Risk-Based Pricing Framework

The most important component of the Business Logic model is the pricing mechanism, where borrower risk estimates are transformed into individualized loan interest rates. As established in the prior Interest Rate Formation Analysis, personal loan pricing is not arbitrary; rather, it is constructed as the sum of several economic and financial components that collectively compensate the lender for all relevant costs and risks associated with issuing the loan. The final pricing equation implemented in the model follows the additive structure:

$$r_i = r_f + \pi + CRP_i + LP + AC + PM$$

where:

- r_i= Final interest rate offered to borrower i
- r_f= Base Rate
- π= Inflation Premium
- CRP_i= Borrower-specific Credit Risk Premium
- LP= Liquidity Premium
- AC= Administrative Cost
- PM= Profit Margin

This structure reflects the theoretical decomposition of lending rates identified in literature and market research.

Base Rate Component

The first pricing layer is the Base Rate, representing the institution's baseline funding cost. This corresponds to the effective Federal Funds Rate, which serves as the benchmark short-term cost of capital in U.S. financial markets.

Based on the market data collected in the research, the Base Rate was calculated at:

$$r_f = 3.64\%$$

This establishes the minimum economic cost at which the lender can deploy capital without incurring losses. No loan can rationally be priced below this rate because doing so would imply lending below the institution's cost of funds.

Inflation Premium

The second component compensates the lender for the erosion of purchasing power over the life of the loan due to inflation. Since personal loans are fixed-payment installment products, lenders remain exposed to future inflation over the duration of the loan contract.

The inflation premium is calibrated using U.S. CPI inflation data:

$$\pi = 2.4\%$$

This ensures that the lender maintains real purchasing power when receiving future repayments.

Credit Risk Premium

The borrower-specific credit risk premium is the only dynamic pricing component and represents the expected financial loss associated with borrower default. This premium is calculated using the standard expected loss framework:

$$CRP_i = PD_i \times LGD$$

where:

- PD_i= Probability of Default of borrower i
- LGD= Loss Given Default

For this model, Loss Given Default was assumed constant at:

$$LGD = 45\%$$

reflecting standard recovery assumptions for unsecured personal lending. Because personal loans lack collateral, lenders face relatively high recovery losses in the event of borrower default, justifying the elevated LGD assumption.

This formula ensures that higher-risk borrowers receive proportionally higher credit spreads.

Liquidity Premium

The liquidity premium compensates the lender for committing capital to a fixed-term, illiquid asset that cannot be easily reallocated elsewhere during the life of the loan. This premium reflects the opportunity cost of tying up lending capital.

Using banking industry benchmark data from Net Interest Margin analysis, the liquidity premium was calibrated at:

$$LP = 3.26\%$$

Administrative Cost Component

Administrative costs account for all operational expenses associated with originating, servicing, and monitoring the loan throughout its lifecycle. This includes underwriting, credit

bureau checks, payment processing, customer servicing, collections, and compliance overhead.

Based on industry estimates cited in the research:

$$AC = 2.00\%$$

Profit Margin Component

Finally, the institution includes a profit margin to ensure long-term economic sustainability and shareholder return. This reflects the minimum profitability target desired after all costs and risks have been covered.

Using FDIC Return on Assets benchmarks:

$$PM = 1.16\%$$

Final Pricing Interpretation

Combining all components yields the complete pricing equation:

$$r_i = 3.64\% + 2.40\% + (PD_i \times 45\%) + 3.26\% + 2.00\% + 1.16\%$$

This formulation ensures that every borrower receives a customized loan rate based on both market/institutional cost conditions, and their individual credit risk profile.

The pricing model therefore mirrors real-world lending practices by integrating macroeconomic conditions, institutional cost structure, and borrower-specific expected losses into one unified pricing mechanism.

Strategic Importance of the Business Logic Layer

By integrating approval rules, borrower segmentation, and risk-based pricing, the Business Logic layer transforms predictive model outputs into actionable lending decisions. Although the machine learning model estimates each borrower's Probability of Default, these predictions must be translated into operational rules to support practical underwriting and pricing strategies.

Through this framework, the institution can control portfolio risk by rejecting borrowers above its risk tolerance threshold, monitor portfolio composition through borrower segmentation, and ensure economically sustainable pricing by incorporating both expected credit losses and institutional operating costs.

In this way, the Business Logic layer completes the credit model by bridging predictive analytics with financial decision-making, creating an underwriting and pricing framework aligned with modern unsecured consumer lending practices.

3. Results & Model Performance

3.1 Model Evaluation

To assess the predictive performance of the proposed credit risk classification model, several evaluation metrics were employed, including Accuracy, Precision, Recall, F1-Score, and ROC-AUC, as recommended in classification modeling literature (James et al., 2021). These metrics provide complementary perspectives regarding the model's predictive capability and discriminatory performance.

Accuracy

Measures the proportion of correctly classified observations relative to the total number of observations:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP denotes true positives, TN true negatives, FP false positives, and FN false negatives.

Precision

Evaluates the proportion of positive predictions that were correctly identified:

$$Precision = \frac{TP}{TP + FP}$$

This metric reflects the reliability of the model when predicting default cases.

Recall

Recall, also referred to as sensitivity, measures the proportion of actual positive cases correctly identified by the model:

$$Recall = \frac{TP}{TP + FN}$$

This metric is especially important in credit risk applications, as it indicates the model's ability to identify borrowers who are likely to default.

F1-Score

Represents the harmonic mean between precision and recall, balancing both metrics into a single indicator:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

This metric is useful when evaluating classification problems with potential imbalance between classes.

ROC-AUC

Finally, ROC-AUC (Receiver Operating Characteristic – Area Under the Curve) measures the model's overall ability to discriminate between positive and negative classes across all classification thresholds. It is based on the relationship between the true positive rate and false positive rate:

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

Higher ROC-AUC values indicate stronger discriminatory power, where 0.50 reflects random classification and 1.00 indicates perfect predictive separation.

3.1.1 Performance Evaluation Results

The model was evaluated using an out-of-sample test dataset corresponding to 20% of the total observations, ensuring unbiased performance assessment on unseen data. Using this methodology, the final model achieved the following results:

Credit Risk Model Performance Metrics	
Metric	**Value**
Test ROC-AUC	0.93
Train ROC-AUC	0.94
Accuracy	84.3%
Precision	81.7%
Recall	86.7%
F1-Score	84.2%

Table IV : Credit Risk Model Performance

These results indicate strong predictive performance, with the model demonstrating robust classification capability and high discriminatory power. Furthermore, the small difference between train and test ROC-AUC suggests that the model generalizes well and presents minimal evidence of overfitting.

3.1.2 ROC - AUC

Training Set

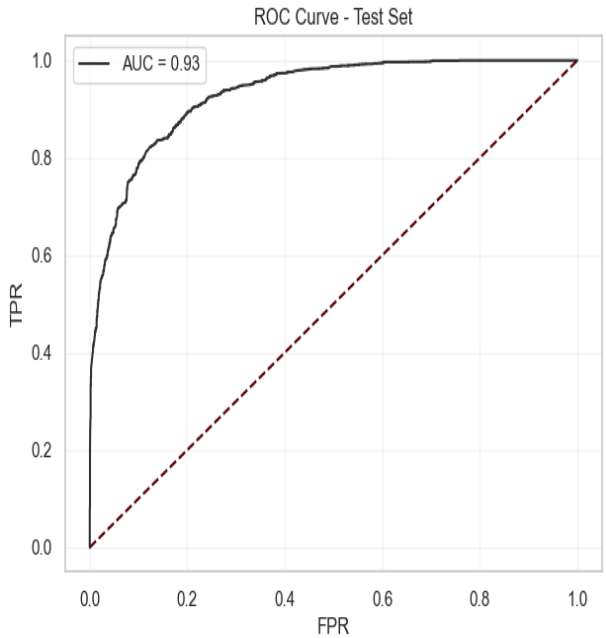

Figure IX. a : Train Set ROC Curve and AUC Performance

Test Set

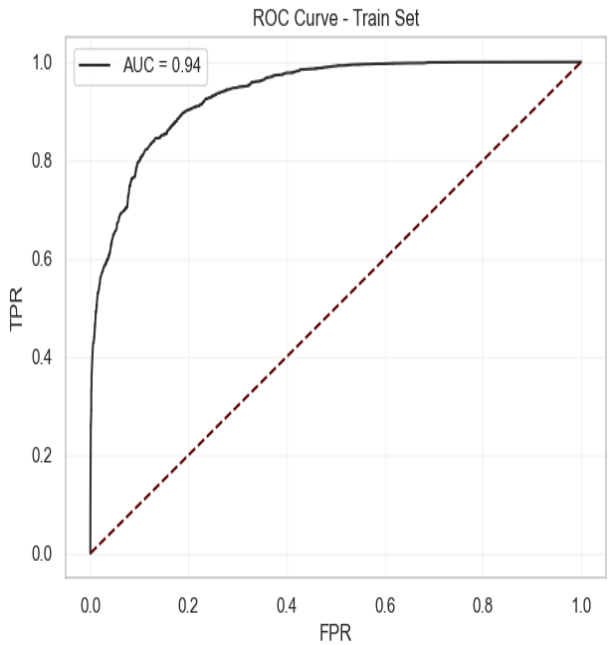

Figure IX. b : Test Set ROC Curve and AUC Performance

As shown in the performance metrics presented in the previous table, the proposed credit risk model achieved strong predictive results across all evaluation criteria. These findings are further supported by the ROC curves above, which visually illustrate the model's discriminatory power for both training and testing datasets.

The training ROC-AUC of 0.94 and testing ROC-AUC of 0.93 indicate that the model effectively distinguishes between defaulting and non-defaulting borrowers while maintaining consistent performance on unseen data. The close similarity between both curves suggests minimal overfitting and good generalization capacity.

Additionally, the clear separation from the diagonal benchmark line confirms that the model performs significantly better than random classification, reinforcing its suitability for borrower risk assessment.

3.1.3 Confusion Matrix

Training Set

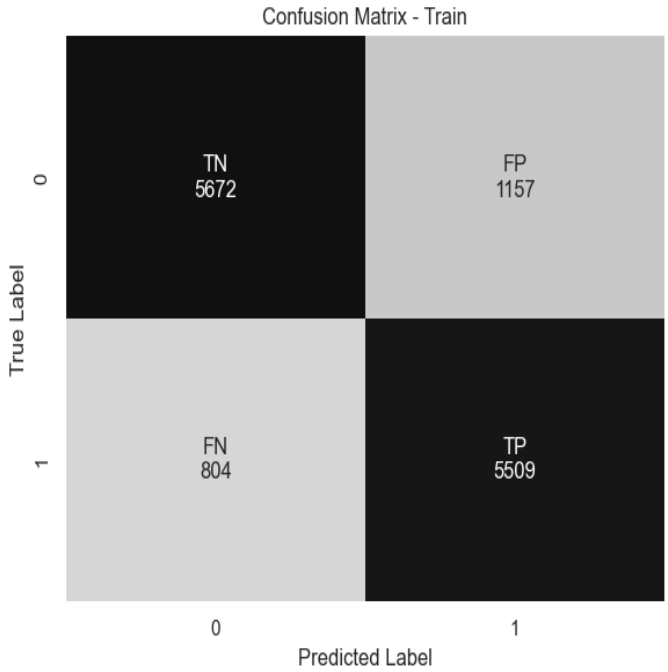

Figure X. a : Confusion Matrix - Train

Test Set

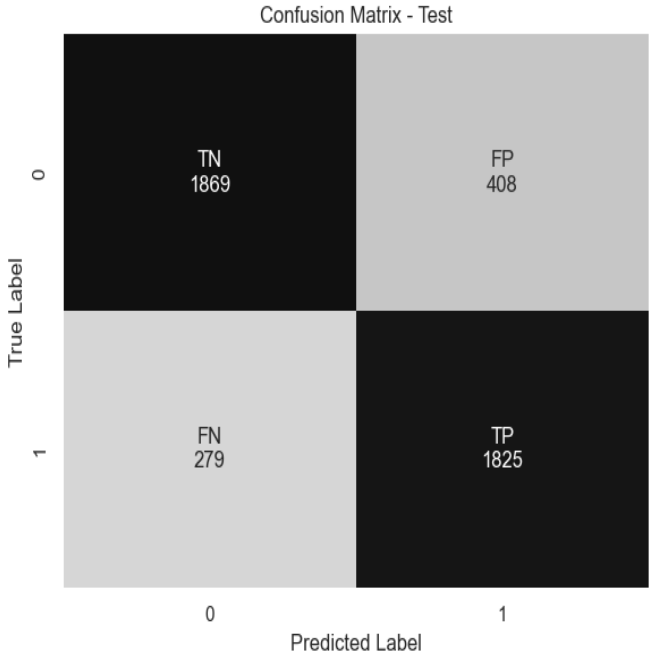

Figure X. b : Confusion Matrix - Test

The confusion matrices for both the training and testing datasets show that the model correctly classifies the majority of observations into their respective categories, indicating strong predictive performance across both borrower groups. Most classifications are concentrated along the diagonal of the matrices, which reflects accurate identification of both defaulting and non-defaulting borrowers.

Some misclassification remains present between the two classes, particularly in cases where borrowers may exhibit financial characteristics similar to those of the opposite category, making them more difficult for the model to distinguish. This overlap is expected in credit risk modeling, as certain borrowers may present borderline risk profiles that do not fit clearly into either classification group.

The relatively balanced distribution of classification outcomes between training and testing datasets suggests that the model maintains stable predictive behavior and generalizes effectively when applied to unseen observations.

3.1.4 Calibration Curve

Training Set

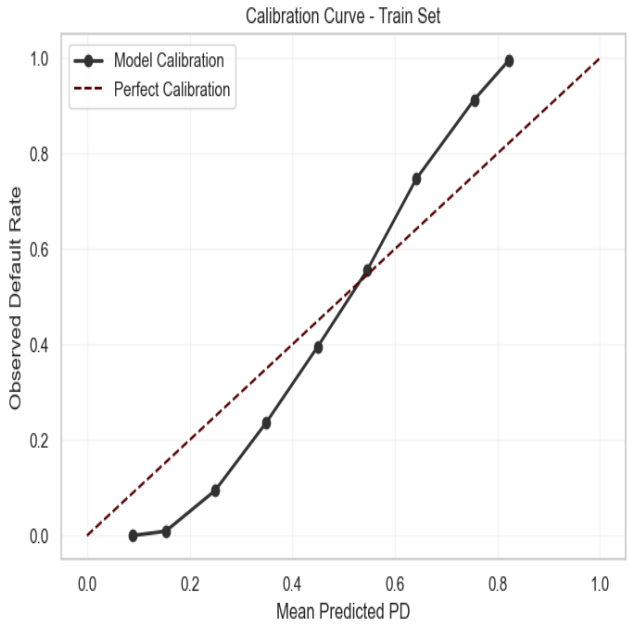

Figure XI. a : Calibration Curve of Predicted Probability of Default (PD)- Train

Test Set

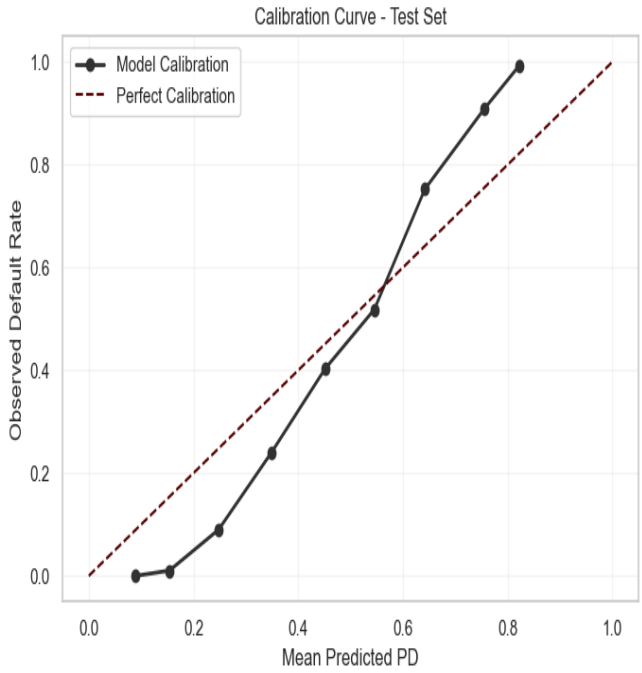

Figure XI. b : Calibration Curve of Predicted Probability of Default (PD)- Test

The calibration curves presented above evaluate the alignment between predicted probabilities of default and observed default frequencies across probability intervals. In both training and testing datasets, the model's calibration curve remains relatively close to the perfect calibration benchmark, indicating that predicted PD values generally correspond well with realized borrower default behavior.

Minor deviations are observed at the lower and upper probability ranges, suggesting slight overestimation in lower-risk segments and underestimation among the highest-risk observations. Nevertheless, the consistency between both datasets indicates stable probability estimation and supports the reliability of the proposed PD outputs for downstream financial applications.

3.1.5 Predicted PD Distribution

Training Set

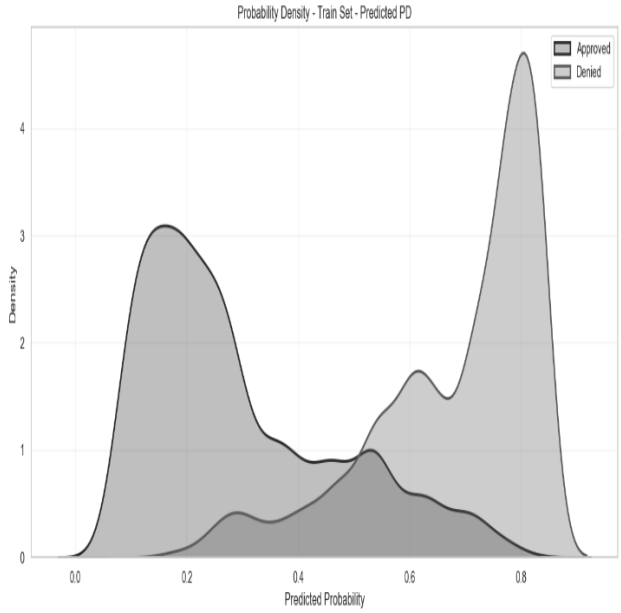

Figure XII. a : Distribution of Predicted Probability of Default by Credit Decision Class - Train

Test Set

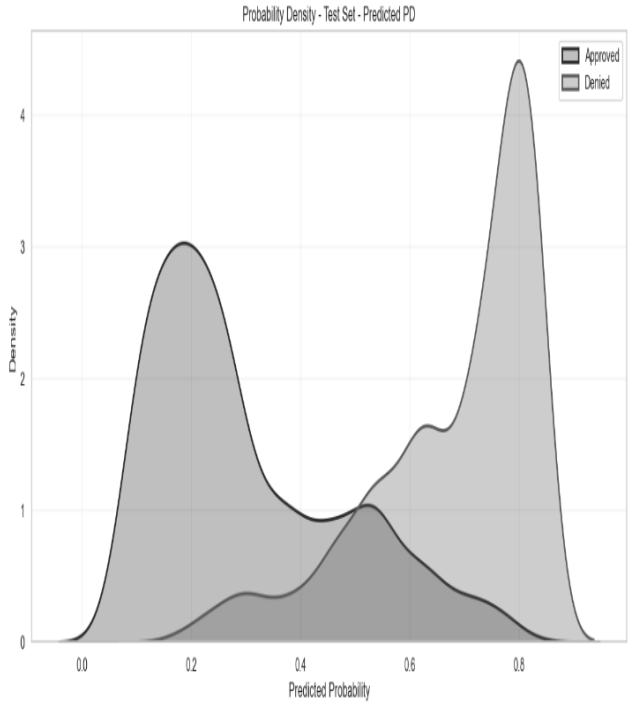

Figure XII. b : Distribution of Predicted Probability of Default by Credit Decision Class - Test

The predicted probability distributions show how the model separates approved and denied applicants along the probability axis. In both the training and test datasets, approved borrowers are primarily concentrated within the lower predicted PD ranges, while denied applicants are more heavily distributed toward higher probability values. This pattern indicates that the model assigns lower default probabilities to applicants with stronger financial profiles and higher probabilities to those exhibiting greater estimated credit risk.

Although some overlap can still be observed between both groups, this is expected given that certain borrowers may present intermediate characteristics that place them near the decision boundary. Nevertheless, the general separation remains consistent across both datasets. As discussed previously, these predicted probabilities represent the estimated likelihood of borrower default and serve as the basis for applicant segmentation within the proposed credit approval framework.

3.1.5.1 Real vs Predicted PD Distributions by Class

Training Set

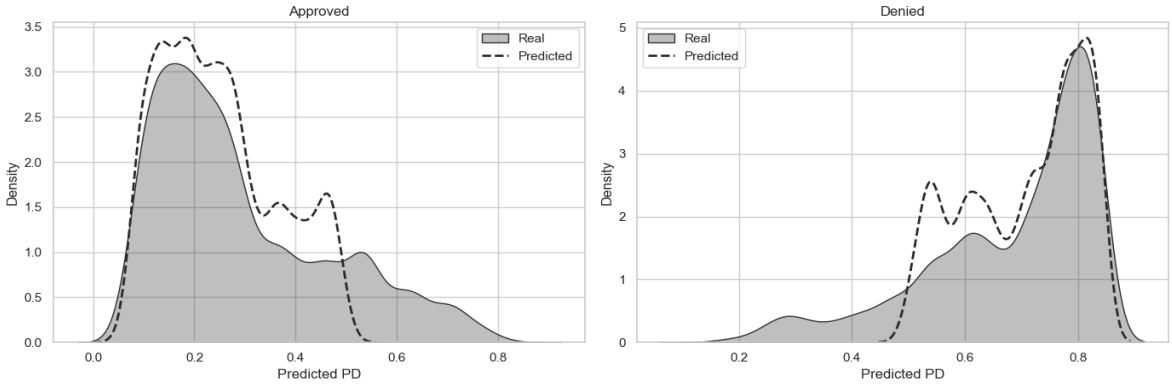

Figure XIII. a : Comparison of Real and Predicted PD Distributions by Credit Decision Outcome

Test Set

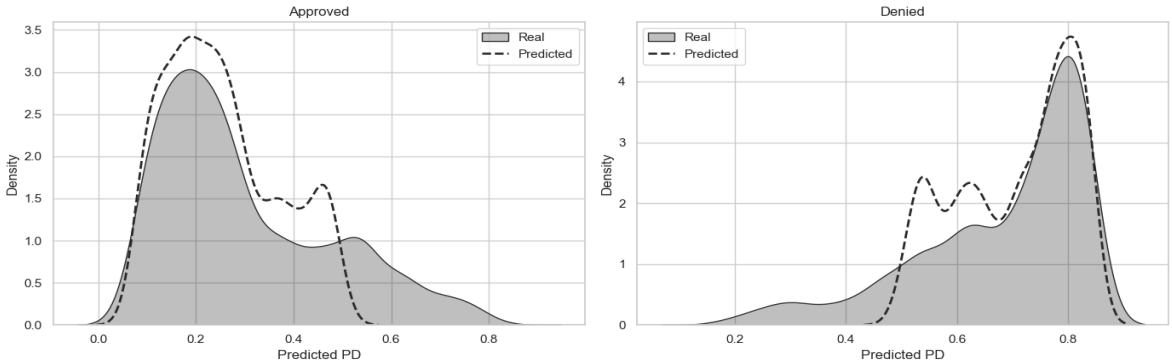

Figure XIII. b : Comparison of Real and Predicted PD Distributions by Credit Decision Outcome

The comparison between the real and predicted PD distributions provides additional insight into how well the proposed model captures the underlying borrower risk structure. In both the training and test datasets, the predicted distributions generally follow the same shape and separation pattern as the real distributions across both borrower classes. Approved applicants remain concentrated in the lower predicted PD region, while denied applicants are clearly shifted toward higher probability values. Although the predicted curves appear slightly more concentrated than the real distributions, the overall alignment between both suggests that the model preserves the relative ranking of borrower risk profiles. Minor deviations between the curves reflect the natural uncertainty present in classification tasks and are consistent with the overlap previously observed between borrowers exhibiting intermediate credit characteristics.

3.1.6 Risk Bucket Analysis

Predicted PD by Risk Bucket

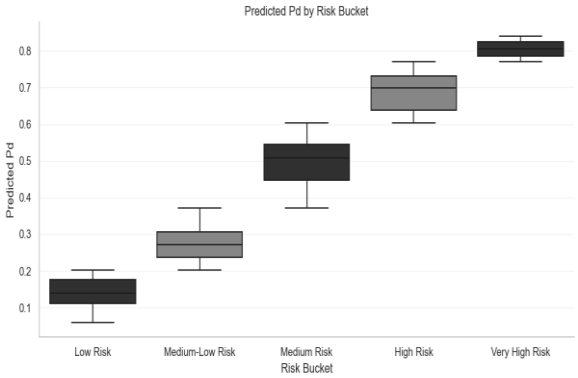

Figure XIV. a : Distribution of Predicted Probability of Default Across Risk Buckets

Expected Loss by Risk Bucket

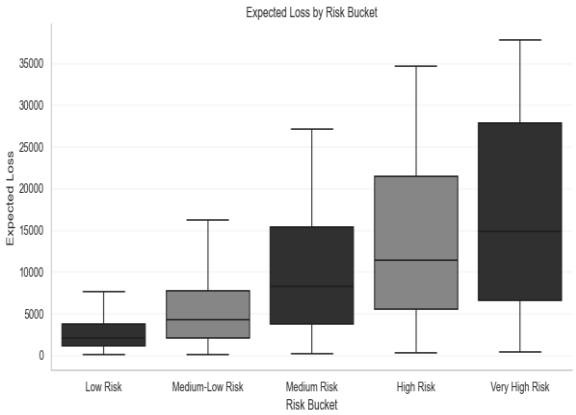

Figure XIV. b : Expected Loss Distribution Across Credit Risk Buckets

Interest Rate by Risk Bucket

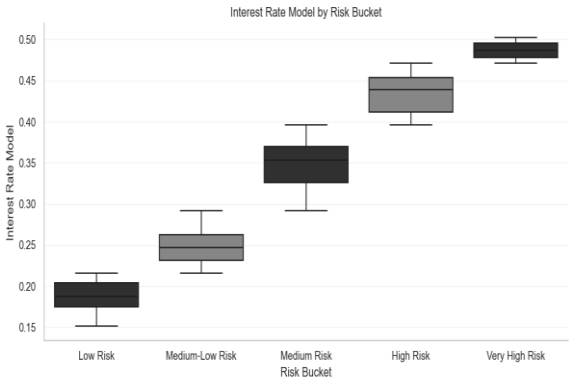

Figure XIV. c : Interest Rate Distribution Across Credit Risk Buckets

The risk bucket analysis illustrates the relationship between borrower risk classification and the financial outputs generated by the proposed model. As borrower risk increases from Low Risk to Very High Risk, predicted probability of default rises consistently across all categories, indicating that the segmentation framework effectively differentiates applicants according to their estimated credit risk.

A similar upward trend is observed in expected loss, where higher-risk buckets are associated with larger projected losses, reflecting the direct relationship between default probability and anticipated lender exposure. Likewise, interest rates increase progressively across each risk bucket, consistent with risk-based pricing principles in which riskier borrowers are charged higher rates to compensate for greater expected losses. These results confirm the internal consistency of the proposed framework between risk assessment, expected loss estimation, and pricing decisions.

Model Limitations & Assumptions & Future Improvements

Despite the strong in-sample and out-of-sample performance achieved, the proposed model operates under several assumptions that limit the scope and generalizability of its outputs in a production environment. The most fundamental limitation is the reliance on a single static dataset which, although realistic in structure, was artificially generated and lacks a temporal dimension. While the dataset provides a useful approximation of lending behavior, it does not fully replicate the complexity of real-world banking portfolios, where borrower performance evolves over time and is significantly influenced by macroeconomic cycles, labor market conditions, and interest rate fluctuations. As a result, the model cannot capture macroeconomic covariate drift, meaning that the statistical relationship between borrower characteristics and default risk may change as economic conditions evolve. This limitation is particularly relevant in unsecured personal lending, where delinquency rates tend to deteriorate during recessionary environments or periods of rising unemployment. Consequently, the estimated PD values should be interpreted as point-in-time measures based on current borrower characteristics rather than through-the-cycle estimates suitable for long-term forecasting or stress testing.

Additional limitations arise from the simplifying assumptions embedded in the risk calculation framework. The use of a constant LGD assumption of 45% across all borrowers ignores the fact that recovery rates may vary materially depending on borrower's profile, collection effectiveness, and economic conditions. Likewise, estimating EAD using the full loan amount assumes that the entire principal remains outstanding at the time of default, without accounting for amortization, early repayments, or partial balance reductions over time. While these assumptions are acceptable for academic implementation and provide a simplified approximation of expected credit loss mechanics, more sophisticated credit risk environments would require borrower-specific and dynamically estimated LGD and EAD models to improve realism and accuracy.

Looking forward, several improvements could significantly enhance the operational maturity of the framework. A natural next step would be the integration of a model lifecycle

management platform such as MLflow, enabling structured experiment tracking, hyperparameter versioning, model registry, and reproducible deployment workflows. Implementing this type of infrastructure would improve governance, scalability, and transparency throughout the model development process. In addition, deploying the model through an interactive dashboard or underwriting interface would allow underwriters and portfolio managers to visualize predicted borrower risk, expected loss estimates, approval decisions, and pricing recommendations in real time, effectively bridging the gap between model outputs and business decision-making.

Finally, the framework could be extended through the incorporation of real-time monitoring systems capable of detecting feature drift, prediction drift, and model performance deterioration over time. Such monitoring would allow institutions to recalibrate or retrain the model as borrower behavior and market conditions evolve. From a broader financial perspective, future development may also include the incorporation of macroeconomic stress testing and scenario analysis, allowing the framework to estimate borrower risk under adverse economic conditions. Although beyond the scope of this academic implementation, these additions would move the model closer toward institutional credit risk practices commonly associated with portfolio stress testing and forward-looking expected loss estimation frameworks.

Final Discussion & Conclusions

The credit risk framework developed throughout this project demonstrates that machine learning methods and traditional financial risk theory can function as complementary tools when integrated into a unified analytical framework. By grounding the predictive model in the traditional components of credit risk measurement—Probability of Default, Loss Given Default, Exposure at Default, and Expected Loss—the proposed system extends beyond simple binary classification and produces outputs that remain interpretable within the context of real-world lending and portfolio management. Rather than serving solely as a predictive algorithm, the model generates borrower-specific risk estimates that can support underwriting decisions, pricing strategies, and portfolio segmentation.

From a lender's perspective, one of the most valuable aspects of the framework is its ability to replicate the logic of risk-based pricing in a structured and data-driven manner. By decomposing the interest rate into its theoretical components—including base rate, inflation premium, credit risk premium, liquidity premium, administrative costs, and profit margin—the pricing model reflects the economic principles underlying unsecured personal loan pricing in the U.S. market. Because the credit risk premium is directly linked to each borrower's estimated Probability of Default, the framework ensures that higher-risk borrowers receive proportionally higher borrowing costs, thereby aligning expected return with expected credit exposure. This creates a pricing methodology that is financially coherent and consistent with modern lending practices.

The model's quantitative performance further supports its usefulness as a practical underwriting tool. Achieving a test ROC-AUC of 0.93, alongside balanced precision and recall metrics, indicates strong discriminatory power and reliable predictive capability across different borrower profiles. Additionally, the risk bucket segmentation demonstrates that predicted PD, expected loss, and assigned interest rates increase consistently across borrower risk categories, confirming internal consistency between the predictive engine and the business logic framework. This monotonic relationship suggests that the model appropriately differentiates borrower quality and produces economically meaningful outputs.

The project successfully delivers a complete end-to-end credit risk framework that integrates predictive analytics, traditional financial risk metrics, and business decision logic into a single cohesive architecture. Beyond its technical implementation, it illustrates how modern lending institutions increasingly rely on the intersection of financial theory, statistical modeling, and machine learning to construct scalable and economically coherent underwriting systems. In this way, the resulting framework not only fulfills the objective of predicting borrower default risk but also demonstrates how quantitative models can support more informed, consistent, and financially sustainable lending decisions.

References

- Board of Governors of the Federal Reserve System. (2025). Federal Reserve issues FOMC statement [Press release]. https://www.federalreserve.gov/newsevents/pressreleases/monetary20251210a.htm
- Board of Governors of the Federal Reserve System. (2026a). Consumer credit — G.19. https://www.federalreserve.gov/releases/g19/current/
- Board of Governors of the Federal Reserve System. (2026b). Selected interest rates — H.15. https://www.federalreserve.gov/releases/h15/
- Board of Governors of the Federal Reserve System. (2026c). Senior loan officer opinion survey on bank lending practices — January 2026. https://www.federalreserve.gov/data/sloos/sloos-202601.htm
- Blokhin, A. (2026). What are some examples of general and administrative expenses? Investopedia. https://www.investopedia.com/ask/answers/073115/what-are-some-examples-of-general-and-administrative-expenses.asp
- Bureau of Labor Statistics. (2026a). Consumer price index — December 2025. U.S. Department of Labor. https://www.bls.gov/news.release/archives/cpi_01132026.htm
- Bureau of Labor Statistics. (2026b). Consumer price index — February 2026. U.S. Department of Labor. https://www.bls.gov/news.release/cpi.nr0.html
- Bureau of Labor Statistics. (2026c). Consumer price index: 2025 in review. U.S. Department of Labor. https://www.bls.gov/opub/ted/2026/consumer-price-index-2025-in-review.htm
- Daily Treasury rates | U.S. Department of the Treasury. (2026, February 13). U.S. Department of The Treasury. https://home.treasury.gov/resource-center/data-chart-center/interest rates/TextView?type=daily_treasury_yield_curve&field_tdr_date_value=2026
- Diette, M. D. (2000, November 1). How do lenders set interest rates on loans? Federal Reserve Bank of Minneapolis. https://www.minneapolisfed.org/article/2000/how-do-lenders-set-interest-rates-on-loans
- CNBC. (2025, December 10). What December's Fed rate cut means for your mortgage, credit card, auto loan, student debt and savings. https://www.cnbc.com/2025/12/10/fed-december-rate-cut-mortgages-loans-savings-impact.html
- Consumer Financial Protection Bureau. (2024). What is risk-based pricing? https://www.consumerfinance.gov/ask-cfpb/what-is-risk-based-pricing-en-767/
- Consumer Financial Protection Bureau. (2025a). What is a personal installment loan? https://www.consumerfinance.gov/ask-cfpb/what-is-a-personal-installment-loan-en-2114/
- Consumer Financial Protection Bureau. (2025b). Consumer credit card market report 2025. https://files.consumerfinance.gov/f/documents/cfpb_consumer-credit-card-market-report_2025.pdf
- Corporate Finance Institute. (n.d.). Liquidity premium. https://corporatefinanceinstitute.com/resources/equities/liquidity-premium/
- Federal Deposit Insurance Corporation. (2025a). Quarterly banking profile: Fourth quarter 2024. https://www.fdic.gov/quarterly-banking-profile/quarterly-banking-profile-q4-2024
- Federal Deposit Insurance Corporation. (2025b). Quarterly banking profile: Second quarter 2025. https://www.fdic.gov/news/speeches/2025/fdic-quarterly-banking-profile-second-quarter-2025
- Federal Deposit Insurance Corporation. (2025c). Quarterly banking profile: First quarter 2025. https://www.fdic.gov/news/speeches/2025/fdic-quarterly-banking-profile-first-quarter-2025
- Federal Deposit Insurance Corporation. (2025d). *FDIC quarterly banking profile: First quarter 2025* [FDIC Quarterly, Vol. 19, No. 2]. https://www.fdic.gov/quarterly-banking-profile/fdic-quarterly-2025-volume-19-number-2.pdf
- Federal Reserve Bank of San Francisco. (2003, August). What is the difference between the real interest rate and the nominal interest rate? https://www.frbsf.org/research-and-insights/publications/doctor-econ/2003/08/real-nominal-interest-rate/
- Federal Reserve Bank of St. Louis. (2026a). Federal funds effective rate [FEDFUNDS]. FRED. https://fred.stlouisfed.org/series/FEDFUNDS
- Federal Reserve Bank of St. Louis. (2026b). Finance rate on personal loans at commercial banks, 24 month loan [TERMCBPER24NS]. FRED. https://fred.stlouisfed.org/series/TERMCBPER24NS

- Federal Reserve Bank of St. Louis. (2026c). Finance rate on consumer installment loans at commercial banks, new autos 48 month loan [TERMCBAUTO48NS]. FRED. https://fred.stlouisfed.org/series/TERMCBAUTO48NS
- Federal Reserve Bank of St. Louis. (2026d). Commercial bank interest rate on credit card plans, all accounts [TERMCBCCALLNS]. FRED. https://fred.stlouisfed.org/series/TERMCBCCALLNS
- Federal Reserve Bank of St. Louis. (2026e). Breakeven inflation rate for US, 5-year [BRMUPL01]. FRED. https://fred.stlouisfed.org/series/BRMUPL01
- Gibson, S. (2025). Revolving credit. Investopedia. https://www.investopedia.com/terms/r/revolvingcredit.asp
- International Monetary Fund. (2026). United States: 2026 Article IV consultation [IMF Country Report]. https://www.imf.org/-/media/files/publications/cr/2026/english/1usaea2026001.pdf
- Kissell, C. (2026, March 31). Personal loan rates today. PrimeRates. https://primerates.com/personal-loans/rates-today/
- Schulz, M. (2026, March 9). Personal loan statistics: 2026. LendingTree. https://www.lendingtree.com/personal/personal-loans-statistics/
- Segal, T. (2026). Profit margin. Investopedia. https://www.investopedia.com/terms/p/profitmargin.asp
- TransUnion. (2025). Q3 2025: Super prime and subprime fueling growth. https://www.transunion.com/blog/q3-2025-super-prime-subprime-fueling-growth
- TransUnion. (2026a). Q4 2025 credit industry insights report: 2026 originations forecast. https://newsroom.transunion.com/q4-2025-ciir/
- TransUnion. (2026b). February 2026 monthly snapshot. https://www.transunion.com/content/dam/transunion/us/business/collateral/report/1-fs/2026-february-monthly-snapshot.pdf
- Vipond, T. (2020). Liquidity premium. Corporate Finance Institute. https://corporatefinanceinstitute.com/resources/equities/liquidity-premium/
- World Bank. (2024). Lending interest rate (%): Metadata glossary. World Development Indicators. https://databank.worldbank.org/metadataglossary/world-development-indicators/series/FR.INR.LEND
- pandas documentation — pandas 3.0.1 documentation. (n.d.). https://pandas.pydata.org/docs/
- NumPy documentation — NumPy v2.4 Manual. (n.d.). https://numpy.org/doc/2.4/
- seaborn: statistical data visualization — seaborn 0.13.2 documentation. (n.d.). https://seaborn.pydata.org/
- SHAP documentation — SHAP latest documentation. (n.d.). https://shap.readthedocs.io/en/latest/
- API Reference — SHAP latest documentation. (n.d.). https://shap.readthedocs.io/en/latest/api.html#explanation
- scikit-learn: machine learning in Python — scikit-learn 1.8.0 documentation. (n.d.). https://scikit-learn.org/stable/index.html
- GradientBoostingClassifier. (n.d.). Scikit-learn documentation. https://scikitlearn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html
- Scikit-learn Developers. (2026). *RandomForestClassifier.* Scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Appendices

Appendix A – Main Script

main.py

```
import pandas as pd
from src.pipeline import CreditPipeline
from src.utils.prints import PrintUtils
from src.visualization.viz import Visualization


def main():
    """
    Execute the end-to-end credit risk modeling pipeline.

    Workflow
    --------
    1. Load dataset from local storage.
    2. Initialize the credit pipeline with the selected model.
    3. Run the pipeline (data preparation, training, prediction,
evaluation).
    4. Display key results.
    5. Generate visualizations for model performance and risk metrics.
    6. (Optional) Save processed results to disk.

    Notes
    -----
    - The pipeline encapsulates the full modeling process, including:
        * Data preprocessing
        * Model training (e.g., Random Forest)
        * Probability of Default (PD) estimation
        * Risk metric computation
    - Ensure that the dataset exists at the specified path before execution.
    """

    # == Load data ==
    data = pd.read_csv("data/dataset.csv")

    # == Initialize pipeline ==
    pipeline = CreditPipeline(data=data, model_name="random_forest")

    # == Run pipeline ==
    results, data_final = pipeline.run()
```

```
    # == Print results (optional) ==
    printer = PrintUtils(data_final)
    printer.print_all(results)

    # == Visualizations ==
    viz = Visualization()
    viz.plot_all(results, data_final)

    # == Save results (optional) ==
    data_final.to_csv("data/results.csv", index=False)

if __name__ == "__main__":
    main()
```

Appendix B – Visualization Script

src/ pipeline.py

```
from src.modeling.model import Model
from src.modeling.config import MODELS_DIR
from src.data.data_splitter import DataSplitter
from src.data.data_preparation import DataPreparation
from src.modeling.business_logic import BusinessLogic
from src.modeling.risk_calculator import RiskCalculator
from src.modeling.model_evaluation import ModelEvaluation


class CreditPipeline:
    """
    End-to-end pipeline for credit risk modeling.

    This pipeline handles data preparation, model training, prediction,
    evaluation, risk metric computation (PD, LGD, EAD, Expected Loss),
    and business decision logic.

    Parameters
    ----------
    data : pd.DataFrame
        Input dataset containing borrower and loan information.

    model_name : str, optional
        Name of the model to be used (default is "logistic").
    """
```

```
    def __init__(self, data, model_name="logistic"):
        self.data = data
        self.model_name = model_name

    def run(self):
        """
        Execute the full credit risk pipeline.

        Steps:
        - Prepare and split data
        - Train model
        - Generate predictions and probabilities
        - Evaluate model performance
        - Compute risk metrics (PD, LGD, EAD, Expected Loss)
        - Apply business rules (decision and risk segmentation)
        - Save trained model

        Returns
        -------
        results : dict
            Dictionary containing evaluation metrics, predictions,
probabilities,
            and AUC scores.

        data : pd.DataFrame
            Dataset enriched with predicted PD, expected loss, decision,
            and risk bucket segmentation.
        """

        # 1. Data preparation
        prep = DataPreparation(self.data)
        X, y = prep.prepare_data()

        # 2. Train-test split
        splitter = DataSplitter()
        X_train, X_test, y_train, y_test = splitter.split(X, y)

        # 3. Model selection
        model = Model.get_model("classification", self.model_name)

        # 4. Training
        model.train(X_train, y_train)

        # 5. Predictions (Test)
        y_pred = model.predict(X_test)
```

```
        y_test_proba = model.predict_proba(X_test)

        # 7. Predictions (Train)
        y_train_proba = model.predict_proba(X_train)
        y_train_pred = model.predict(X_train)

        # 8. Evaluation
        evaluator = ModelEvaluation()

        results = evaluator.evaluate_full(
            y_train=y_train,
            y_train_pred=y_train_pred,
            y_train_proba=y_train_proba,
            y_test=y_test,
            y_test_pred=y_pred,
            y_test_proba=y_test_proba
        )

        # 9. PD calculation (full dataset)
        risk = RiskCalculator(lgd=0.45)
        pd_values = risk.calculate_pd(model, X)
        self.data["predicted_pd"] = pd_values

        # 10. Risk metrics
        ead = risk.calculate_ead(self.data)
        lgd = risk.calculate_lgd(self.data)

        self.data["expected_loss"] = risk.calculate_expected_loss(
            pd_values, lgd, ead
        )

        # 11. Business logic
        logic = BusinessLogic(threshold=0.4, LGD=0.45 , rf=0.0364,
                 inflation_premium=0.024, liquidity_premium=0.0326,
admin_cost=0.02, profit_margin=0.0116)
        self.data["decision"] = logic.credit_decision(pd_values)
        self.data["risk_bucket"] = logic.risk_buckets(pd_values)
        self.data["interest_rate_model"] =
logic.calculate_interest_rate(pd_values)

        # 12. Save model
        model.save_model(f"{self.model_name}.pkl", MODELS_DIR)

        return results, self.data
```

Appendix C – Data Preparation Script

src/data/data_preparation.py

```
import pandas as pd


class DataPreparation:
    """
    Class responsible for preparing raw credit data for modeling.

    This class handles the preprocessing pipeline required before
    model training, including:

    - Separating the target variable from predictor variables.
    - Removing non-predictive or unnecessary columns.
    - Encoding categorical variables using one-hot encoding.
    - Selecting the final subset of features used for modeling.

    Attributes
    ----------
    data : pd.DataFrame
        Raw input dataset copied from the original source.

    SELECTED_FEATURES : list[str]
        List of final predictor variables selected for model training.
    """

    SELECTED_FEATURES = [
        'credit_score',
        'debt_to_income_ratio',
        'credit_history_years',
        'years_employed',
        'payment_to_income_ratio',
        'loan_intent_Education',
        'delinquencies_last_2yrs',
        'defaults_on_file'
    ]

    def __init__(self, data: pd.DataFrame) -> None:
        """
        Initialize the DataPreparation object.
```

```
        Parameters
        ----------
        data : pd.DataFrame
            Raw dataset containing borrower information and loan outcomes.
        """
        self.data = data.copy()

    def prepare_data(self) -> tuple[pd.DataFrame, pd.Series]:
        """
        Execute the full preprocessing pipeline for model input.

        This method performs the following steps:

        1. Extracts the target variable ('loan_status').
        2. Removes irrelevant or non-feature columns.
        3. Encodes categorical variables into dummy/indicator variables.
        4. Filters the dataset to retain only selected model features.

        Returns
        -------
        tuple[pd.DataFrame, pd.Series]
            A tuple containing:

            - X : Processed feature matrix ready for modeling.
            - y : Target variable representing loan default status.

        Raises
        ------
        ValueError
            If one or more selected features are not found after
preprocessing.
        """
        # Target
        y = self.data['loan_status']

        # Features
        X = self.data.drop(
            columns=["Unnamed: 0", "loan_status", "interest_rate"]
        )

        # Encoding
        X = pd.get_dummies(X, drop_first=True)

        # Validate selected features
        missing = set(self.SELECTED_FEATURES) - set(X.columns)
```

```
        if missing:
            raise ValueError(
                f"Missing selected features after preprocessing: {missing}")

        # Feature selection
        X = X[self.SELECTED_FEATURES]

        return X, y
```

Appendix D – Data Splitter Script

src/data/data_splitter.py

```
import pandas as pd
from typing import Tuple
from sklearn.model_selection import train_test_split


class DataSplitter:
    """
    Class responsible for splitting data into training and testing sets.

    This class encapsulates the train-test split logic to ensure
    reproducibility and consistency across the pipeline.
    """

    def __init__(self, test_size: float = 0.25, random_state: int = 42) ->
None:
        """
        Initialize the data splitter.

        Parameters
        ----------
        test_size : float, optional
            Proportion of the dataset to include in the test split (default
= 0.25).
        random_state : int, optional
            Seed used for random number generation to ensure
reproducibility.
        """
        self.test_size = test_size
```

```
        self.random_state = random_state

    def split(self, X: pd.DataFrame, y: pd.Series) -> Tuple[pd.DataFrame,
pd.DataFrame, pd.Series, pd.Series]:
        """
        Split the dataset into training and testing sets.

        Parameters
        ----------
        X : pd.DataFrame
            Feature matrix.
        y : pd.Series
            Target variable.

        Returns
        -------
        Tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series]
            X_train, X_test, y_train, y_test
        """
        return train_test_split(X, y, test_size=self.test_size,
random_state=self.random_state , stratify=y)
```

Appendix E – Base Model Script

src/modeling/base_model.py

```
import numpy as np
import pandas as pd
from abc import ABC, abstractmethod


class BaseModel(ABC):
    """
    Abstract base class for machine learning models.

    Defines the interface that all models must implement.
    """

    @abstractmethod
    def train(self, X: pd.DataFrame, y: pd.Series) -> None:
        """
        Train the model.
```

```
        Parameters
        ----------
        X : pd.DataFrame
            Feature matrix.
        y : pd.Series
            Target variable.
        """
        pass

    @abstractmethod
    def predict(self, X: pd.DataFrame) -> np.ndarray:
        """
        Generate predictions.

        Parameters
        ----------
        X : pd.DataFrame
            Feature matrix.

        Returns
        -------
        np.ndarray
            Predicted labels.
        """
        pass

    @abstractmethod
    def predict_proba(self, X: pd.DataFrame) -> np.ndarray:
        """
        Generate predicted probabilities (for classification models).

        Parameters
        ----------
        X : pd.DataFrame
            Feature matrix.

        Returns
        -------
        np.ndarray
            Predicted probabilities.
        """
        pass
```

Appendix F – Business Logic Script

src/modeling/business_logic.py

```
import pandas as pd


class BusinessLogic:
    """
    Implements core business rules for credit decisioning,
    customer risk segmentation, and pricing-related logic.

    Attributes
    ----------
    threshold : float
        Probability of Default (PD) cutoff used to approve or reject
applications.
        Clients with PD below this threshold are approved.
    """

    def __init__(self, threshold=0.4, LGD=0.45 , rf=0.0364,
                 inflation_premium=0.024, liquidity_premium=0.0326,
admin_cost=0.02, profit_margin=0.0116):
        self.threshold = threshold
        self.LGD = LGD
        self.rf = rf
        self.inflation_premium = inflation_premium
        self.liquidity_premium = liquidity_premium
        self.admin_cost = admin_cost
        self.profit_margin = profit_margin

    def credit_decision(self, pd_values: pd.Series) -> pd.Series:
        """
        Generate binary credit decisions based on PD values.

        Parameters
        ----------
        pd_values : pd.Series
            Series containing Probability of Default (PD) estimates for each
client.

        Returns
        -------
        pd.Series
```

```
            Binary decision for each client:
            - 1 : Approved (PD < threshold)
            - 0 : Rejected (PD >= threshold)

        Notes
        -----
        This rule represents a simple risk-based cutoff strategy.
        In real-world applications, this threshold may vary depending on
        portfolio strategy, regulatory constraints, or economic conditions.
        """
        return (pd_values < self.threshold).astype(int)

    def risk_buckets(self, pd_values: pd.Series, q=5) -> pd.Series:
        """
        Segment clients into risk buckets based on PD quantiles.

        Parameters
        ----------
        pd_values : pd.Series
            Series containing Probability of Default (PD) values.
        q : int, default=5
            Number of quantile-based buckets (e.g., 5 = quintiles).

        Returns
        -------
        pd.Series
            Categorical series indicating the risk bucket assigned to each
client.

        Notes
        -----
        - Buckets are created using quantiles (equal-sized groups).
        - Lower buckets correspond to lower risk (lower PD).
        - Useful for portfolio segmentation, pricing strategies, and
monitoring.
        - If there are too many duplicate PD values, pd.qcut may raise an
error.
        """
        return pd.qcut(pd_values, q=q)

    def calculate_interest_rate(self, pd_values: pd.Series) -> pd.Series:
        """
        Calculate interest rate based on risk (PD).

        Higher PD -> higher interest rate.
```

```
        """

        risk_premium = pd_values * self.LGD

        return self.rf + self.inflation_premium + risk_premium +
self.liquidity_premium + self.admin_cost + self.profit_margin
```

Appendix G – Classification Model Script

src/modeling/classification_model.py

```
import joblib
import pandas as pd
from src.modeling.config import MODEL_CONFIG
from src.modeling.base_model import BaseModel
from xgboost.sklearn import XGBClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier


class ClassificationModel(BaseModel):
    """
    Wrapper class for classification models with support for multiple
algorithms.

    This class provides a unified interface for training, prediction,
    probability estimation, and model persistence. The model configuration
    is retrieved from a centralized configuration dictionary.

    Supported models:
    - Logistic Regression
    - Random Forest
    - XGBoost
    """

    def __init__(self, model_name: str) -> None:
        """
        Initialize the classification model based on the specified model
name.

        Parameters
        ----------
        model_name : str
```

```
            - Name of the model to initialize.

            - Supported values are:
            'logistic', 'random_forest', 'xgboost'.

        Raises
        ------
        ValueError
            If the provided model_name is not supported.
        """
        self.model_name = model_name
        self.config = MODEL_CONFIG.get(model_name, {})

        if model_name == "logistic":
            self.model = LogisticRegression(**self.config)

        elif model_name == "random_forest":
            self.model = RandomForestClassifier(**self.config)

        elif model_name == "xgboost":
            self.model = XGBClassifier(**self.config)

        else:
            raise ValueError("Modelo no soportado")

    def train(self, X: pd.DataFrame, y: pd.Series) -> None:
        """
        Train the classification model.

        Parameters
        ----------
        X : pd.DataFrame
            Feature matrix used for training.
        y : pd.Series
            Target variable (binary classification).
        """
        self.model.fit(X, y)

    def predict(self, X: pd.DataFrame) -> pd.Series:
        """
        Generate class predictions for the input data.

        Parameters
        ----------
        X : pd.DataFrame
```

```
            Feature matrix for prediction.

        Returns
        -------
        pd.Series
            Predicted class labels.
        """
        return self.model.predict(X)

    def predict_proba(self, X: pd.DataFrame) -> pd.Series:
        """
        Generate predicted probabilities for the positive class (PD).

        Parameters
        ----------
        X : pd.DataFrame
            Feature matrix for prediction.

        Returns
        -------
        pd.Series
            Probability of the positive class (default probability).
        """
        return self.model.predict_proba(X)[:, 1]

    def save_model(self, filename: str, models_dir) -> None:
        """
        Save the trained model to disk using joblib.

        The method stores the model along with metadata such as the model
name
        and configuration.

        Parameters
        ----------
        filename : str
            Name of the file to save the model (e.g., 'model.pkl').
        models_dir : Path
            Directory where the model will be stored.
        """
        models_dir.mkdir(parents=True, exist_ok=True)
        path = models_dir / filename

        joblib.dump({
            "model": self.model,
```

```
            "model_name": self.model_name,
            "config": self.config
        }, path)

    def load_model(self, filename: str, models_dir) -> None:
        """
        Load a previously saved model from disk.

        Parameters
        ----------
        filename : str
            Name of the file containing the saved model.
        models_dir : Path
            Directory where the model is stored.
        """
        path = models_dir / filename
        data = joblib.load(path)

        self.model = data["model"]
        self.model_name = data["model_name"]
        self.config = data["config"]
```

Appendix H – Configuration Script

src/modeling/config.py

```
"""
Configuration module for the credit risk modeling project.

This module centralizes:
- Hyperparameters for different machine learning models
- Project directory paths

This design allows easy modification of model settings and ensures
consistency across the pipeline.
"""

from pathlib import Path


MODEL_CONFIG = {
    "logistic": {
        "max_iter": 1000,
```

```
        "solver": "lbfgs"
    },

    "random_forest": {
        "n_estimators": 1000,
        "max_depth": 2,
        "random_state": 42,
        "class_weight": "balanced",
        "n_jobs": -1
    },

    "xgboost": {
        "objective" : "binary:logistic",
        "n_estimators": 250,
        "max_depth": 2,
        "learning_rate": 0.01,
        "subsample": 0.7,
        "colsample_bytree": 0.8,
        "eval_metric": "logloss",
        "random_state": 42
    }
}


# Root directory of the project
BASE_DIR = Path(__file__).resolve().parent.parent

# Directory where trained models are stored
MODELS_DIR = BASE_DIR / "models"
```

Appendix I – Model Evaluation Script

src/modeling/model_evaluation.py

```
from sklearn.metrics import f1_score, precision_score, recall_score,
roc_auc_score, accuracy_score, confusion_matrix


class ModelEvaluation:
    """
    Class for evaluating machine learning models.
```

```
    This class supports evaluation for classification tasks using
    common performance metrics such as ROC-AUC, accuracy, precision,
    recall, F1-score, and confusion matrix.

    The design allows future extension to regression tasks.
    """

    def __init__(self, task_type: str = "classification") -> None:
        """
        Initialize the evaluation object.

        Parameters
        ----------
        task_type : str, optional
            Type of task to evaluate. Currently supports:
            - 'classification'
            Default is 'classification'.
        """
        self.task_type = task_type

    def evaluate(self, y_true, y_pred, y_pred_proba=None) -> dict:
        """
        Evaluate model predictions based on the specified task type.

        Parameters
        ----------
        y_true : array-like
            True target values.
        y_pred : array-like
            Predicted class labels.
        y_pred_proba : array-like, optional
            Predicted probabilities for the positive class.
            Required for ROC-AUC calculation.

        Returns
        -------
        dict
            Dictionary containing evaluation metrics.

        Raises
        ------
        ValueError
            If the task_type is not supported.
        """
        if self.task_type == "classification":
```

```
            return self._evaluate_classification(y_true, y_pred,
y_pred_proba)

        # Future extension for regression
        # elif self.task_type == "regression":
        #     return self._evaluate_regression(y_true, y_pred)

        else:
            raise ValueError("Type of evaluation not supported")

    def _evaluate_classification(self, y_true, y_pred, y_pred_proba) ->
dict:
        """
        Compute classification performance metrics.

        Parameters
        ----------
        y_true : array-like
            True labels.
        y_pred : array-like
            Predicted labels.
        y_pred_proba : array-like
            Predicted probabilities for the positive class.

        Returns
        -------
        dict
            Dictionary with the following metrics:
            - roc_auc
            - accuracy
            - precision
            - recall
            - f1_score
            - confusion_matrix
        """
        results = {
            "roc_auc": roc_auc_score(y_true, y_pred_proba),
            "accuracy": accuracy_score(y_true, y_pred),
            "precision": precision_score(y_true, y_pred),
            "recall": recall_score(y_true, y_pred),
            "f1_score": f1_score(y_true, y_pred),
            "confusion_matrix": confusion_matrix(y_true, y_pred)
        }

        return results
```

```
    # Regression
        '''
    def _evaluate_regression(self, y_true, y_pred) -> dict:

        from sklearn.metrics import mean_squared_error, r2_score

        results = {
            "rmse": mean_squared_error(y_true, y_pred, squared=False),
            "r2": r2_score(y_true, y_pred)
        }

        return results
        '''

    def evaluate_full(self, y_train,y_train_pred ,y_train_proba, y_test,
y_test_pred, y_test_proba):
        """
        Perform full train/test evaluation and return structured results.

        Parameters
        ----------
        y_train : pd.Series
            True training labels.

        y_train_pred : np.ndarray
            Predicted training labels.

        y_train_proba : np.ndarray
            Predicted training probabilities.

        y_test : pd.Series
            True testing labels.

        y_test_pred : np.ndarray
            Predicted testing labels.

        y_test_proba : np.ndarray
            Predicted testing probabilities.

        Returns
        -------
        dict
            Dictionary containing train/test metrics and raw predictions.
        """
```

```
        results = {}

        test_metrics = self._evaluate_classification(
            y_test,
            y_test_pred,
            y_test_proba
        )

        results.update({
            f"test_{k}": v for k, v in test_metrics.items()
        })

        train_metrics = self._evaluate_classification(
            y_train,
            y_train_pred,
            y_train_proba
        )

        results.update({
            f"train_{k}": v for k, v in train_metrics.items()
        })

        results["y_test"] = y_test
        results["y_pred"] = y_test_pred
        results["y_prob"] = y_test_proba

        results["y_train"] = y_train
        results["y_train_pred"] = y_train_pred
        results["y_train_prob"] = y_train_proba

        return results
```

Appendix J – Model Script

src/modeling/model.py

```
from src.modeling.classification_model import ClassificationModel


class Model:
    """
    Factory class responsible for creating machine learning model instances.
```

```
    This class abstracts the model instantiation process and allows
    dynamic selection of models based on the task type and model name.

    Currently supports:
    - Classification models

    Future extensions may include regression models.
    """

    @staticmethod
    def get_model(task_type: str, model_name: str):
        """
        Create and return a model instance based on task type and model
name.

        Parameters
        ----------
        task_type : str
            Type of machine learning task. Supported values:
            - 'classification'

        model_name : str
            Name of the model to instantiate. Supported values:
            - 'logistic'
            - 'random_forest'
            - 'xgboost'

        Returns
        -------
        object
            An instance of the requested model.

        Raises
        ------
        ValueError
            If the task type or model name is not supported.
        """

        if task_type == "classification":
            allowed_models = ["logistic", "random_forest", "xgboost"]

            if model_name not in allowed_models:
                raise ValueError(f"Model '{model_name}' not supported")

            return ClassificationModel(model_name)
```

```
        # IF WE WANT TO ADD REGRESSION MODELS IN THE FUTURE, WE CAN DO IT
HERE
            '''
        elif task_type == "regression":
            return RegressionModel(config)
            '''

        else:
            raise ValueError("Invalid model type")
```

Appendix K – Risk Calculator Script

src/modeling/risk_calculator.py

```
import pandas as pd


class RiskCalculator:
    """
    Computes key credit risk metrics used in credit modeling.

    This class provides methods to estimate:
    - Probability of Default (PD)
    - Loss Given Default (LGD)
    - Exposure at Default (EAD)
    - Expected Loss (EL)

    These metrics are fundamental for risk assessment,
    pricing, and portfolio management.
    """

    def __init__(self, lgd: float = 0.45):
        """
        Initialize the RiskCalculator.

        Parameters
        ----------
        lgd : float, default=0.45
            Assumed Loss Given Default.
            Represents the proportion of exposure lost if a default occurs.
        """
        self.lgd = lgd
```

```python
    def calculate_pd(self, model, X: pd.DataFrame) -> pd.Series:
        """
        Estimate the Probability of Default (PD) using a trained model.

        Parameters
        ----------
        model : object
            Trained classification model with a `predict_proba` method
            (e.g., Logistic Regression, Random Forest, XGBoost).
        X : pd.DataFrame
            Feature matrix used for prediction.

        Returns
        -------
        pd.Series
            Estimated PD values for each observation.

        Notes
        -----
        - Assumes that the model outputs probabilities where column 1
          corresponds to the probability of default.
        - Typically, PD = model.predict_proba(X)[:, 1]
        """
        return model.predict_proba(X)

    def calculate_ead(self, data: pd.DataFrame) -> pd.Series:
        """
        Estimate Exposure at Default (EAD).

        Parameters
        ----------
        data : pd.DataFrame
            Dataset containing loan information.

        Returns
        -------
        pd.Series
            Exposure values at default for each observation.

        Notes
        -----
        - Uses `loan_amount` as a proxy for EAD.
        - In more advanced models, EAD may include credit conversion factors
(CCF).
```

```
        """
        return data["loan_amount"]

    def calculate_lgd(self, data: pd.DataFrame) -> pd.Series:
        """
        Estimate Loss Given Default (LGD).

        Parameters
        ----------
        data : pd.DataFrame
            Dataset (not directly used in this implementation).

        Returns
        -------
        pd.Series
            LGD values for each observation.

        Notes
        -----
        - This implementation assumes a constant LGD.
        - In practice, LGD may depend on collateral, seniority, or recovery
rates.
        """
        return pd.Series(self.lgd, index=data.index)

    def calculate_expected_loss(self, pd: pd.Series, lgd: pd.Series, ead:
pd.Series) -> pd.Series:
        """
        Compute Expected Loss (EL).

        Parameters
        ----------
        pd : pd.Series
            Probability of Default.
        lgd : pd.Series
            Loss Given Default.
        ead : pd.Series
            Exposure at Default.

        Returns
        -------
        pd.Series
            Expected Loss for each observation.

        Notes
```

```
        -----
        Expected Loss is defined as:

            EL = PD × LGD × EAD

        This metric is widely used in:
        - Credit pricing
        - Risk provisioning (e.g., IFRS 9)
        - Capital allocation (Basel framework)
        """
        return pd * lgd * ead
```

Appendix L – Prints Script

src/utils/prints.py

```
import pandas as pd


class PrintUtils:
    """
    Utility class for displaying model outputs, risk metrics,
    and portfolio analyses in a structured and readable format.

    This class centralizes all printing logic to ensure consistency
    across evaluation, risk segmentation, and pricing analysis.
    """

    def __init__(self, data: pd.DataFrame) -> None:
        """
        Initialize the PrintUtils class.

        Parameters
        ----------
        data : pd.DataFrame
            Final dataset containing model outputs and risk metrics,
            such as predicted PD, expected loss, risk buckets, and
decisions.
        """
        self.data = data

    def print_model_results(self, results: dict) -> None:
        print("\n=== Model Evaluation ===")
```

```
        metrics_to_show = [
            "test_roc_auc",
            "train_roc_auc",
            "test_accuracy",
            "test_precision",
            "test_recall",
            "test_f1_score",
        ]

        for key in metrics_to_show:
            print(f"{key}: {results[key]}")


    def print_sample(self) -> None:
        """
        Display a sample of the dataset with key outputs.

        Notes
        -----
        Shows the first rows including:
        - predicted_pd : estimated probability of default
        - expected_loss : calculated expected loss
        - decision : credit approval decision
        """
        print("\n=== Sample Results ===")
        print(self.data[['predicted_pd', 'expected_loss',
'decision']].head())

    def print_risk_analysis(self) -> None:
        """
        Display average risk metrics by risk bucket.

        Notes
        -----
        Aggregates the following metrics by risk segment:
        - predicted_pd
        - expected_loss

        Useful for understanding how risk is distributed across segments.
        """
        print("\n=== Risk Buckets (PD & Expected Loss) ===")
        print(
            self.data.groupby('risk_bucket')[
                ['predicted_pd', 'expected_loss']].mean()
```

```
        )

    def print_pricing_analysis(self) -> None:
        """
        Display relationship between risk and pricing.

        Notes
        -----
        Aggregates:
        - predicted_pd
        - interest_rate

        by risk bucket to analyze whether pricing is aligned with risk
levels.
        """
        print("\n=== PD vs Interest Rate ===")
        print(
            self.data.groupby('risk_bucket')[
                ['predicted_pd', 'interest_rate_model']].mean()
        )

    def print_all(self, results: dict) -> None:
        """
        Execute all available print methods.

        Parameters
        ----------
        results : dict
            Model evaluation metrics.

        Notes
        -----
        This method provides a complete overview of:
        - Model performance
        - Sample predictions
        - Risk segmentation
        - Pricing consistency
        """
        self.print_model_results(results)
        self.print_sample()
        self.print_risk_analysis()
        self.print_pricing_analysis()
```

Appendix M – Visualization Script

src/visualization/viz.py

```
import warnings
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.calibration import calibration_curve
from matplotlib.colors import LinearSegmentedColormap
from sklearn.metrics import auc, roc_curve, confusion_matrix as
sk_confusion_matrix

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)

sns.set_theme(style="whitegrid", palette="Greys_r")
plt.rcParams["figure.figsize"] = (12, 6)
plt.rcParams["figure.dpi"] = 100


class Visualization:
    """
    Visualization utilities for evaluating and analyzing credit risk models.

    Provides methods for plotting model performance metrics, distributions,
    and relationships between key variables such as PD, expected loss, and
interest rate.
    """

    def plot_roc_curve(self, y_test, y_prob, name="Test Set"):
        """
        Plot the Receiver Operating Characteristic (ROC) curve.

        Parameters
        ----------
        y_test : array-like
            True binary labels.

        y_prob : array-like
            Predicted probabilities for the positive class.

        name : str, optional
            Label for the dataset (e.g., "Train Set", "Test Set").
```

```
        """
        fpr, tpr, _ = roc_curve(y_test, y_prob)
        roc_auc = auc(fpr, tpr)

        plt.figure(figsize=(7, 5))
        plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}", color="#313131")
        plt.plot([0, 1], [0, 1], linestyle='--', color="#580000")
        plt.xlabel("FPR")
        plt.ylabel("TPR")
        plt.title(f"ROC Curve - {name}")
        plt.legend()
        plt.grid(alpha=0.2)
        plt.show()

    def plot_confusion_matrix(self, y_test, y_pred , name = ""):
        """
        Plot a confusion matrix with labeled cells.

        Parameters
        ----------
        y_test : array-like
            True labels.

        y_pred : array-like
            Predicted class labels.
        """
        cm = sk_confusion_matrix(y_test, y_pred)

        labels = np.array([["TN", "FP"], ["FN", "TP"]])
        annotated = np.empty_like(cm).astype(str)

        for i in range(cm.shape[0]):
            for j in range(cm.shape[1]):
                annotated[i, j] = f"{labels[i, j]}\n{cm[i, j]}"

        cmap = LinearSegmentedColormap.from_list(
            "custom",
            ["#d6d6d6", "#101010"]
        )

        plt.figure(figsize=(6, 5))
        sns.heatmap(cm, annot=annotated, fmt="", cmap=cmap, cbar=False)

        plt.title(f"Confusion Matrix - {name}")
        plt.xlabel("Predicted Label")
```

```
        plt.ylabel("True Label")
        plt.tight_layout()
        plt.show()

    def plot_scatter(self, data, hue, x, y):
        """
        Plot a scatter plot for two variables with optional hue grouping.

        Parameters
        ----------
        data : pd.DataFrame
            Dataset containing variables.

        hue : str
            Column name used for color grouping.

        x : str
            Column name for x-axis.

        y : str
            Column name for y-axis.
        """
        plt.figure(figsize=(7, 5))

        sns.scatterplot(
            data=data,
            x=x,
            y=y,
            hue=hue,
            palette=["#313131", "#505050"],
            marker="^",
            s=70,
            alpha=0.8
        )

        plt.xlabel(x.replace('_', ' ').title())
        plt.ylabel(y.replace('_', ' ').title())
        plt.title(
            f"{y.replace('_', ' ').title()} vs {x.replace('_', '
').title()}")
        plt.legend(title=hue)
        plt.grid(alpha=0.2)
        plt.show()

    def plot_bar(self, data, x, y):
```

```
        """
        Plot a bar chart of average values grouped by a categorical
variable.

        Parameters
        ----------
        data : pd.DataFrame
            Dataset containing variables.

        x : str
            Grouping variable.

        y : str
            Numerical variable to average.
        """
        data.groupby(x)[y].mean().plot(kind='bar')

        plt.title(
            f"Average {y.replace('_', ' ').title()} by {x.replace('_', '
').title()}")
        plt.xlabel(x.replace('_', ' ').title())
        plt.ylabel(f"Average {y.replace('_', ' ').title()}")
        plt.xticks(rotation=0)
        plt.grid(alpha=0.2)
        plt.show()

    def plot_distribution(self, x, dataset_name, var_name):
        """
        Plot the distribution (KDE) of a numerical variable.

        Parameters
        ----------
        x : array-like
            Data values.

        dataset_name : str
            Dataset label.

        var_name : str
            Variable name for labeling.
        """
        plt.figure(figsize=(10, 5))

        sns.kdeplot(
            x=x,
```

```
            fill=True,
            alpha=0.3,
            linewidth=2
        )

        plt.title(f"{dataset_name} {var_name}")
        plt.xlabel(var_name)
        plt.ylabel("Density")
        plt.grid(alpha=0.2)
        plt.show()

    def plot_boxplot(self, data, x, y, hue=None, order=None):
        """
        Plot a boxplot for a numerical variable grouped by categories.

        Parameters
        ----------
        data : pd.DataFrame
            Dataset containing variables.

        x : str
            Categorical variable.

        y : str
            Numerical variable.

        hue : str, optional
            Additional grouping variable.

        order : list, optional
            Order of categories for x-axis.
        """

        data = data.copy()

        if x == "risk_bucket":
            bucket_labels = [
                "Low Risk",
                "Medium-Low Risk",
                "Medium Risk",
                "High Risk",
                "Very High Risk"
            ]

            data[x] = data[x].astype(str)
```

```
            unique_buckets = sorted(data[x].unique())

            mapping = dict(zip(unique_buckets, bucket_labels))

            data[x] = data[x].map(mapping)

            order = bucket_labels

        plt.figure(figsize=(10, 5))

        ax = sns.boxplot(
            data=data,
            x=x,
            y=y,
            hue=hue,
            order=order,
            palette=["#303030", "#868686"],
            showfliers=False,
            linewidth=1.2
        )

        ax.set_title(
            f"{y.replace('_', ' ').title()} by {x.replace('_', '
').title()}"
        )

        ax.set_xlabel(x.replace('_', ' ').title())
        ax.set_ylabel(y.replace('_', ' ').title())

        plt.grid(axis='y', alpha=0.2)

        sns.despine()

        plt.tight_layout()

        plt.show()

    def plot_probability_density(self , y_test , proba, model_name: str =
"Modelo", class_names: tuple[str, str, str] = ("Denied","Approved")) ->
None:
        """
        Plot kernel density estimation of predicted probabilities.

        This visualization shows how probability estimates differ
```

```
        across the true classes.

        Parameters
        ----------
        y_test : array-like
            True class labels.
        proba : np.ndarray
            Predicted class probabilities.
        model_name : str
            Model name used in the plot title.
        class_names : tuple
            Names of the classes.
        """
        y_test = np.array(y_test)

        plt.figure()

        if proba.ndim == 1:

            sns.kdeplot(
                proba[y_test == 0],
                fill=True,
                alpha=0.3,
                linewidth=2,
                label=class_names[0]
            )

            sns.kdeplot(
                proba[y_test == 1],
                fill=True,
                alpha=0.3,
                linewidth=2,
                label=class_names[1]
            )

        plt.title(f"Probability Density - {model_name}")
        plt.xlabel("Predicted Probability")
        plt.ylabel("Density")
        plt.legend()
        plt.grid(alpha=0.2)
        plt.tight_layout()
        plt.show()


    def plot_real_vs_predicted_pd(self,
```

```
            predicted_pd: np.ndarray,
            true_labels: pd.Series | np.ndarray,
            predicted_labels: np.ndarray,
            dataset_name: str = "Train"
        ) -> None:
        """
        Compare real and predicted PD distributions for Approved/Denied
classes.

        Parameters
        ----------
        predicted_pd : np.ndarray
            Predicted probability of default values.
        true_labels : array-like
            True borrower labels.
        predicted_labels : np.ndarray
            Predicted borrower labels.
        dataset_name : str
            Dataset identifier (Train/Test).
        """

        results_df = pd.DataFrame({
            "Predicted_PD": predicted_pd,
            "Real_Class": true_labels,
            "Predicted_Class": predicted_labels
        })

        label_names = {
            0: "Approved",
            1: "Denied"
        }

        fig, axes = plt.subplots(1, 2, figsize=(14, 5))

        for class_id in range(2):

            real_dist = results_df[
                results_df["Real_Class"] == class_id
            ]["Predicted_PD"]

            pred_dist = results_df[
                results_df["Predicted_Class"] == class_id
            ]["Predicted_PD"]

            sns.kdeplot(
```

```
                real_dist,
                ax=axes[class_id],
                label="Real",
                fill=True,
                alpha=0.3
            )

            sns.kdeplot(
                pred_dist,
                ax=axes[class_id],
                label="Predicted",
                linestyle="--",
                linewidth=2
            )

            axes[class_id].set_title(
                f"{label_names[class_id]}"
                )

            axes[class_id].set_xlabel("Predicted PD")
            axes[class_id].set_ylabel("Density")
            axes[class_id].legend()

        plt.tight_layout()
        plt.show()



    def plot_calibration_curve(
        self,
        y_true,
        predicted_pd,
        dataset_name="Test Set",
        n_bins=10
    ):
        """
        Plot calibration curve comparing predicted PD vs observed default
rate.

        Parameters
        ----------
        y_true : array-like
            True binary labels.
        predicted_pd : array-like
            Predicted probabilities of default.
```

```
        dataset_name : str
            Dataset label.
        n_bins : int
            Number of bins for calibration.
        """

        prob_true, prob_pred = calibration_curve(
            y_true,
            predicted_pd,
            n_bins=n_bins,
            strategy='uniform'
        )

        plt.figure(figsize=(7, 5))

        plt.plot(
            prob_pred,
            prob_true,
            marker='o',
            linewidth=2,
            color="#313131",
            label="Model Calibration"
        )

        plt.plot(
            [0, 1],
            [0, 1],
            linestyle='--',
            color="#580000",
            label="Perfect Calibration"
        )

        plt.xlabel("Mean Predicted PD")
        plt.ylabel("Observed Default Rate")
        plt.title(f"Calibration Curve - {dataset_name}")
        plt.legend()
        plt.grid(alpha=0.2)

        plt.tight_layout()
        plt.show()



    def plot_all(self, results, data):
        """
```

```
        Generate a full set of visualizations for model evaluation and
analysis.
        """

        # ==========================================
        # 1. MODEL DISCRIMINATION PERFORMANCE
        # ==========================================

        # ROC Curves
        self.plot_roc_curve(
            results['y_test'],
            results['y_prob'],
            name="Test Set"
        )

        self.plot_roc_curve(
            results['y_train'],
            results['y_train_prob'],
            name="Train Set"
        )

        # Confusion Matrices
        self.plot_confusion_matrix(
            results['y_test'],
            results['y_pred'],
            name="Test"
        )

        self.plot_confusion_matrix(
            results['y_train'],
            results['y_train_pred'],
            name="Train"
        )

        # ==========================================
        # 2. PROBABILITY / PD VALIDATION
        # ==========================================

        # Calibration Curves
        self.plot_calibration_curve(
            results['y_train'],
            results['y_train_prob'],
            dataset_name="Train Set"
        )
```

```
        self.plot_calibration_curve(
            results['y_test'],
            results['y_prob'],
            dataset_name="Test Set"
        )

        # PD Density by Class
        self.plot_probability_density(
            results['y_train'],
            results['y_train_prob'],
            model_name="Train Set - Predicted PD",
            class_names=("Approved", "Denied")
        )

        self.plot_probability_density(
            results['y_test'],
            results['y_prob'],
            model_name="Test Set - Predicted PD",
            class_names=("Approved", "Denied")
        )

        # Real vs Predicted PD Alignment
        self.plot_real_vs_predicted_pd(
            predicted_pd=results['y_train_prob'],
            true_labels=results['y_train'],
            predicted_labels=results['y_train_pred'],
            dataset_name="Train Set"
        )

        self.plot_real_vs_predicted_pd(
            predicted_pd=results['y_prob'],
            true_labels=results['y_test'],
            predicted_labels=results['y_pred'],
            dataset_name="Test Set"
        )

        # ==========================================
        # 3. RISK SEGMENTATION / FINANCIAL OUTPUTS
        # ==========================================

        # Risk Bucket Means
        self.plot_bar(
            data,
            x='risk_bucket',
            y='predicted_pd'
```

```
        )

        self.plot_bar(
            data,
            x='risk_bucket',
            y='expected_loss'
        )

        # Risk Bucket Distributions
        self.plot_boxplot(
            data,
            x='risk_bucket',
            y='predicted_pd'
        )

        self.plot_boxplot(
            data,
            x='risk_bucket',
            y='expected_loss'
        )

        self.plot_boxplot(
            data,
            x='risk_bucket',
            y='interest_rate_model'
        )
```